

SECTION 4 INSTRUCTION SET

This section describes the MC68881/MC68882 (FPCP) instruction set in detail, using the Freescale assembly language syntax and notation. As an introduction, a summary of the instruction set is presented, followed by a detailed description of each instruction. Also, included at the end of this section is a listing of the binary patterns of all of the instructions and an opcode map summary for use by assembler and compiler writers.

4.1 INSTRUCTION DESCRIPTION CONVENTIONS

The instruction set is discussed in this section using functional grouping and the following notation:

B, W, L	The same size codes as all M68000 Family processors; specifies a signed integer data type (two's complement) of byte (8 bits), word (16 bits), or long word (32 bits)
S	Single precision real data format (32 bits)
D	Double precision real data format (64 bits)
X	Extended precision real data format (96 bits, 16 bits unused)
P	Packed BCD real data format (96 bits, 12 bytes)
FPm, FPn	One of eight floating-point data registers
FPcr	One of the three floating-point system control registers (FPCR, FPSR, or FPIAR)
<ea>	Any valid MC68020/MC68030 (MPU) address mode
k	A two's-complement signed integer (−64 to +17) that specifies the format of a number to be stored in the packed decimal format
ccc	An index into the FPCP constant ROM
<list>	A list of floating-point data registers or control registers
<label>	A relative label used by an assembler to calculate a displacement

4.2 INSTRUCTION GROUPS

The following paragraphs briefly describe each instruction group along with tables showing the Freescale syntax for each instruction. The FPCP instructions can be separated into the following groups:

- Data Movement
- Dyadic Operations
- Monadic Operations
- Program Control
- System Control

This group of instructions includes those that load or store the user-visible configuration of the FPCP and that move operands into, between, or out of the floating-point data registers. Data format conversion functions are also implicitly supported since all external data formats are converted to extended precision for internal storage, and vice versa. Operations to move the system control registers into and out of the FPCP are also provided. The move constant ROM (FMOVECR) instruction allows floating-point data registers to be loaded quickly with commonly used constants such as π , e , 0.0, 1.0, etc. Table 4-1 summarizes the data movement instructions that are available and the operand data formats supported.

Table 4-1. Data Movement Operations

Instruction	Operand Syntax	Operand Format	Operation
FMOVE	FPm,FPn <ea>,FPn FPm,<ea> FPm,<ea>{#k} FPm,<ea>{Dn} <ea>,FPcr FPcr,<ea>	X B,W,L,S,D,X,P B,W,L,S,D,X P P L L	source ∇ destination
FMOVECR	#ccc,FPn	X	ROM constant ∇ FPn
FMOVEM	<ea>,{list} ¹ <ea>,Dn {list} ¹ ,<ea> Dn,<ea>	L,X X L,X X	Listed register ∇ destination source ∇ listed registers

NOTE:

1. The register list may include any combination of the eight floating-point data registers, or it may contain any combination of the three control register FPCR, FPSR, and FPIAR. If the register list mask resides in a main processor data register, only floating-point data registers may be specified.

4.2.2 Dyadic Operations

The dyadic floating-point instructions provide several arithmetic functions that require two input operands such as add, subtract, multiply, and divide. For these operations, the first operand may be located in memory, in an integer data register, or in a floating-point data register, and the second operand is always contained in a floating-point data register. The results of the operation are stored in the register specified as the second operand. With two exceptions, all operations support any data format and are performed to extended precision. The exceptions are the single precision multiply and divide instructions (FSGLMUL and FSGLDIV). These instructions support any precision inputs, but return results accurate only to single precision. These instructions provide very high-speed operations by sacrificing accuracy. The general format of the dyadic instructions is given in Table 4-2; the available operations are listed in Table 4-3.

Table 4-2. Dyadic Operation Format

Instruction	Operand Syntax	Operand Format	Operation
F(<dop>)	<ea>,FPn FPm,FPn	B,W,L,S,D,X,P X	FPn {function} source ∇ FPn

where:

<dop> is any one of the dyadic operation specifiers.

Table 4-3. Dyadic Operations

Instruction	Function
FADD	Add
FCMP	Compare
FDIV	Divide
FMOD	Modulo Remainder
FMUL	Multiply
FREM	IEEE Remainder
FSCALE	Scale Exponent
FSGLDIV	Single Precision Divide
FSGLMUL	Single Precision Multiply
FSUB	Subtract

4.2.3 Monadic Operations

The monadic floating-point instructions provide several arithmetic functions that require only one input operand. Unlike the integer counterparts to these function (e.g., NEG <ea>), a source and a destination may be specified. The operation is performed on the source operand, and the result is stored in the destination, which is always a floating-point data register. When the source is not a floating-point data register, all data formats are supported; the data format is always extended precision for register-to-register operations. The general format of these instructions is shown in Table 4-4, and the available operations are listed in Table 4-5. The form of the simultaneous sine and cosine instruction is given in Table 4-6.

Table 4-4. Monadic Operation Format

Instruction	Operand Syntax	Operand Format	Operation
F(<mop>)	<ea>,FPn FPm,FPn FPn	B,W,L,S,D,X,P X X	source ♦ function ♦ FPn FPn ♦ function ♦ FPn

where:

<mop> is any one of the monadic operation specifiers.

Table 4-5. Monadic Operations

Instruction	Function
FABS	Absolute Value
FACOS	Arc Cosine
FASIN	Arc Sine
FATAN	Arc Tangent
FATANH	Hyperbolic Arc Tangent
FCOS	Cosine
FCOSH	Hyperbolic Cosine
FETOX	e^x
FETOM1	$e^x - 1$
FGETEXP	Extract Exponent
FGETMAN	Extract Mantissa
FINTE	Extract Integer Part
FINTRZ	Extract Integer Part, Rounded-to-Zero

Instruction	Function
FLOGN	$\ln(x)$
FLOGNP1	$\ln(x + 1)$
FLOG10	$\log_{10}(x)$
FLOG2	$\log_2(x)$
FNEG	Negate
FSIN	Sine
FSINH	Hyperbolic Sine
FSQRT	Square Root
FTAN	Tangent
FTANH	Hyperbolic Tangent
FTENTOX	10^x
FTWOTOX	2^x

Table 4-6. Dual Monadic Operation Format

Instruction	Operand Syntax	Operand Format	Operation
FSINCOS	(ea),FPc:FPs FPm,PFc:FPs	B,W,L,S,D,X,P X	SIN(source) ∇ FPs; COS(source) ∇ FPC

4.2.4 Program Control Operations

The program control instructions provide a means of affecting program flow based on conditions present in the floating-point status register after any operation that sets the condition codes. In addition to allowing direct control of program flow with the branch conditionally (FPcc) and the decrement and branch conditionally (FDBcc) instructions, the set conditionally (FScc) instruction allows the user to set a Boolean variable based on the floating-point condition codes as an intermediate result in the evaluation of a complex Boolean equation. Also included is a test operand instruction (FTST) that sets the floating-point condition codes for use by the other program and system control instructions, and a no operation instruction (FNOP) that may be used to force synchronization of the FPCP with the main processor. Table 4-7 summarizes the program control instructions that are available.

Table 4-7. Program Control Operations

Instruction	Operand Syntax	Operand Format	Operation
FBcc	(label)	W,L	If Condition True, Then PC + d ∇ PC
FDBcc	Dn,(label)	W	If Condition True, Then No Operation; Else Dn - 1 ∇ Dn; If Dn $\neq$ -1 The PC + d ∇ PC
FNOP	None	None	No Operation
FScc	(ea)	B	If Condition True, The 1's ∇ Destination Else 0's ∇ Destination
FTST	(ea) FPn	B,W,L,S,D,X,P X	Set FPSR Condition Codes

The FPCP supports 32 conditional tests that are separated into two groups — 16 that cause an exception if an unordered condition is present when the conditional test is attempted, and 16 that do not cause an exception if an unordered condition is present. (An unordered condition occurs when an input to an arithmetic operation is a NAN.) Table 4-8 lists the 32 condition code mnemonics along with the conditional test function. Refer to **4.4 CONDITIONAL TEST DEFINITIONS** for a detailed description of the conditional equation used by each test.

Table 4-8. Conditional Test Mnemonics

Exception on Unordered		No Exception on Unordered	
GE	Greater Than or Equal	OGE	Ordered Greater Than or Equal
GL	Greater Than or Less Than	OGL	Ordered Greater Than or Less Than
GLE	Greater Than or Less	OR	Ordered
GT	Greater Than	OGT	Ordered Greater Than

Table 4-8. Conditional Test Mnemonics (Continued)

Exception on Unordered		No Exception on Unordered	
LE	Less Than or Equal	OLE	Ordered Less Than or Equal
LT	Less Than	OLT	Ordered Less Than
NGE	Not (Greater Than or Equal)	UGE	Unordered or Greater Than Equal
NGL	Not (Greater Than or Less Than)	UEQ	Unordered or Equal
NGLE	Not (Greater Than or Less Than or Equal)	UN	Unordered
NGT	Not Greater Than	UGT	Unordered or Greater Than
NLE	Not (Less Than or Equal)	ULE	Unordered or Less Than or Equal
NLT	Not Less Than	ULT	Unordered or Less Than
SEQ	Signaling Equal	EQ	Equal
SNE	Signaling Not Equal	NE	Not Equal
SF	Signaling Always False	F	Always False
ST	Signaling Always True	T	Always True

4.2.5 System Control Operations

The system control instructions communicate with the operating system via a conditional trap instruction (FTRAPcc) and save or restore (FSAVE or FRESTORE) the nonuser visible portion of the FPCP during context switches in a virtual memory or other type of multi-tasking system. The conditional trap instruction uses the same conditional tests as the program control instructions and allows an optional 16- or 32-bit immediate operand to be included as part of the instruction for passing parameters to the operating system. Table 4-9 summarizes the system control instructions.

Table 4-9. System Control Operations

Instruction	Operand Syntax	Operand Size	Operation
FRESTORE	(ea)	None	State Frame $\leftrightarrow$ Internal Registers
FSAVE	(ea)	None	Internal Registers $\leftrightarrow$ State Frame
FTRAPcc	None #xxx	None W,L	If Condition True, Then Take Exception

4.3 COMPUTATIONAL ACCURACY

Whenever an attempt is made to represent a real number in a binary format of finite precision, there is a possibility that the number cannot be represented exactly; this is commonly referred to as round-off error. Furthermore, when two inexact numbers are used in a calculation, the error present in each number is reflected and possibly aggravated in the result.

One of the major reasons that the *IEEE Standard for Binary Floating-Point Arithmetic* (ANSI/IEEE Std. 754-1985) was developed is to define the error bounds for calculation of binary

ing-point values so that all machines conforming to the standard produce the same results for an operation. The operation must meet the following conditions:

1. Same input values,
2. Same rounding mode, and
3. Same precision.

The IEEE standard specifies not only the format of data items, but also defines:

1. The maximum allowable error that may be introduced during a calculation, and
2. The manner in which rounding of the result is performed.

However, the IEEE specification defines only the operation of some of the instructions supported by the FPCP; those not specified by the IEEE standard are described in detail in the following paragraphs. The following paragraphs discuss the accuracy of the calculations performed by the FPCP, grouping them as follows:

1. The IEEE specified operations and nontranscendental functions,
2. The transcendental functions, and
3. The IEEE specified conversions between binary and decimal real formats.

4.3.1 Arithmetic Instructions

The *IEEE Specification for Binary Floating-Point Arithmetic* specifies that the following operations must be supported for each data format: add, subtract, multiply, divide, remainder, square root, integer part, and compare. Conversions between the various data formats are also required. In addition to these arithmetic functions, the FPCP also supports the nontranscendental operations of: absolute value, get exponent, get mantissa, negate, modulo remainder, scale exponent, and test. Since the IEEE specification defines the error bounds to which all calculations are performed, the result obtained by any conforming machine can be predicted exactly for a particular precision and rounding mode. The error bound defined by the IEEE specification is one-half unit in the last place of the destination data format in the round-to-nearest mode and one unit in the last place in the other rounding modes.

The FPCP performs all calculations using a 67-bit mantissa for the intermediate results. The three bits beyond the precision of the extended format allow the FPCP to perform all calculations as if to infinite precision and then round the result to the desired precision before storing it in the destination. By performing calculations in this manner, the final result is always correct for the specified destination data format before rounding is performed (unless an overflow or underflow error occurs). The specified rounding operation then produces a number that is as close as possible to the infinitely precise intermediate value and is still representable in the selected precision. An example of how the 67-bit mantissa allows the FPCP to meet the error bound of the IEEE specification is as follows:

	Mantissa	1	g	r	s
Intermediate Result:	x.x.....x00	1	0	0	(Tie Case)
Round-to-Nearest Result:	x.x.....x00				

In this case, the least significant bit (1) of the rounded result is not incremented, even though the guard bit (g) is set in the intermediate result. The IEEE standard specifies that tie cases should be handled in this manner. Assuming that the destination data format is



ded, if the difference between the infinitely precise intermediate result and the round-nearest result is calculated, the relative difference is 2^{-64} (the value of the guard bit). This error is equal to one-half of the value of the least significant bit and is the worst-case error that can be introduced when using the round-to-nearest mode. Thus, the term one-half unit in the last place correctly identifies the error bound for this operation. This error specification is the relative error present in the result; the absolute error bound is equal to $2^{\text{exponent}} \times 2^{-64}$. An example of the error bound for the other rounding modes is as follows:

	Mantissa	1	g	r	s
Intermediate Result:	x.x.....x00		1	1	1
Round-to-Zero Result:	x.x.....x00				

4

In this case, the difference between the infinitely precise result and the rounded result is the error bound for this operation is not more than one unit in the last place. For all of the arithmetic operations, these error bounds are met by the FPCP, thus providing accurate and repeatable results.

4.3.2 Transcendental Instructions

The IEEE specification does not define the error bound to which transcendental (except square root) functions are to be performed. In this context, the transcendental functions are all of those operations not mentioned in the previous paragraphs (i.e., the trigonometric, hyperbolic, logarithmic, and exponential instructions). Due to the highly recursive nature of the algorithms used to calculate these functions, the round-off error in the input operands to a function, combined with the limited precision of the FPCP ALU, do not allow the calculation of a result with the same error limit as the arithmetic functions. However, these operations are quite accurate given the constraint of using an ALU with a finite precision of 67 bits. In general, the worst-case accuracy of any transcendental function is one unit in the last place of double precision (which is equal to 4096 units in the last place of extended precision). The typical error bound for these instructions is approximately 64 units in the last place of extended precision. The following example illustrates the significance of this error bound:

	Mantissa
Correct Result:	x.x.....x00000000
FPCP Calculated Result:	x.x.....x01000000

In this case, the relative difference between the correct result and the result calculated by the FPCP is 2^{-57} (assuming an extended precision result), which is 26 times the value of the least significant bit. This difference corresponds to an error of 64 units in the last place.

Note that the transcendental functions perform limited checking for special case input values such as boundary conditions. For example, the exponential functions check for a zero input value, but do not check for exact integer values. Thus, raising a number to an exact integer value may not produce an exact result (e.g., the instruction FTENTOX #1,FP0 does not produce an extended precision value of exactly 10.0), and the INEX2 bit in the FPSR may be set even if an exact result is produced.

The IEEE standard does not specify the format of the decimal real representation used by any conforming machine, but it does define the error bounds for conversions between decimal and the single- and double-precision binary formats. Thus, such conversions always produce consistently rounded results, and those results are predictable and repeatable on any conforming system. However, it is not always possible to perform an exact conversion between these data formats, due to the limited precision of the numbers and the different radices of the values. The error bound for these conversions is 0.97 unit in the last digit of the destination precision for the round-to-nearest mode; and 1.47 units in the last digit of the destination precision for the other rounding modes. When an input conversion cannot produce an exact result, the FPCP sets the INEX1 bit in the FPSR exception byte. This indication allows for special handling of these conversion errors that is separate from the handling of other types of inaccurate results. When an output conversion cannot produce an exact result, the INEX2 bit is set.

The packed decimal data format supported by the FPCP allows the representation of double-precision binary number in a decimal form, in accordance with the IEEE specification. When a packed decimal number is converted to extended precision, the result is always in range although the conversion may be inexact. The result is within range because the magnitudes of the exponent and mantissa of a packed decimal number are less than the largest values representable in the extended precision format. Refer to **6.1.8 Inexact Result on Decimal Input** for a description of the handling of inaccurate decimal-to-binary conversions.

When an extended precision number is converted to packed decimal, the result may be a number that cannot be represented exactly, or a number that is too large to be represented with a three-digit exponent. When this type of conversion is performed, the k factor specified is used to locate the decimal rounding boundary. If the magnitude of the rounded decimal result exponent exceeds 999, the FPCP signals an operand error and calculates a fourth exponent digit, which is included in the destination operand (see Figure 3-11 for the position of the fourth digit). Refer to **6.1.7 Inexact Result** for a description of the handling of inaccurate binary-to-decimal conversions.

Note that the error bounds specified by the IEEE standard apply only to conversions of values in the range of the double-precision format. The error bound for conversions by the FPCP of extended precision values which cannot be represented in double precision is significantly larger. Software must be provided to convert such extended precision values to decimal. The conversion must generate decimal results with an error bound analogous to those specified in the IEEE standard for double-precision values. The software envelope must utilize a super extended precision to achieve such error bounds.

Note that the binary to/from decimal conversions performed by the FPCP utilize the on-chip ROM values of powers of 10 for speed and accuracy, thus allowing exact conversions in many cases (particularly for values that are exact powers of 10).

4.4 CONDITIONAL TEST DEFINITIONS

The FPCP provides a very simple mechanism for performing conditional tests of the result of any arithmetic floating-point operation. First, the condition code bits in the FPSR are set or cleared at the end of any arithmetic operation or move operation to a single floating-point data register. The condition code bits are always set consistently based on the result



operation. Second, the FPCP provides 32 conditional tests that are supported in are by the M68000 Family coprocessor interface. This mechanism allows conditional instructions that test floating-point conditions to be coded in exactly the same way as the integer conditional instructions. The evaluation of the conditional test by the FPCP is performed automatically. The combination of the consistent setting of the condition code bits and the simple programming of conditional instructions gives the MC68020/MC68030 and FPCP combination a very flexible, high-performance method of altering program flow based on floating-point results.

One important programming consideration is that the inclusion of the NAN data type in the IEEE floating-point number system requires each conditional test to include the NAN condition code bit in its Boolean equation. Because a comparison of a NAN with anything is unordered (i.e., it is impossible to determine if a NAN is bigger or smaller than an in-range number), the compare instruction sets the NAN condition code bit when an unordered compare is attempted. All arithmetic instructions also set the NAN bit if the result of an operation is a NAN. The conditional instructions interpret the NAN condition code bit equal to one as the unordered condition.

The inclusion of the unordered condition in floating-point branches destroys the familiar trichotomy relationship (greater than, equal, less than) that exists for integers. For example, the opposite of floating-point branch greater than (FBGT) is not floating-point branch less than or equal (FBLE). Rather, the opposite condition is floating-point branch not greater than (FBNGT). If the result of the previous instruction was unordered, FBNGT is true; whereas, both FBGT would be false since unordered fails both of these tests (and sets BSUN). Compiler programmers should be particularly careful of the lack of trichotomy in the floating-point branches since it is common for compilers to invert the sense of conditions.

In the following paragraphs, the conditional tests are described in three main categories:

1. IEEE nonaware tests,
2. IEEE aware test, and
3. Miscellaneous.

The set of IEEE nonaware tests is best used:

1. When porting a program from a system that does not support the IEEE standard to a conforming system, or
2. When generating high-level language code that does not support IEEE floating-point concepts (i.e., the unordered condition).

When using the set of IEEE nonaware tests, the user receives a BSUN exception whenever a branch is attempted and the NAN condition code bit is set, unless the branch is an FBEQ or an FBNE. If the BSUN trap is enabled in the FPCR register, the exception causes a trap. Therefore, the IEEE nonaware program is interrupted if something unexpected occurs.

The IEEE aware branch set should be used in programs that contain ordered and unordered conditions by compilers and programmers who are knowledgeable of the IEEE standard. Since the ordered or unordered attribute is explicitly included in the conditional test, the BSUN bit is not set in the status register EXC byte when the unordered condition occurs.

For all the conditional tests in the following table, except EQ and NE, set the BSUN bit in the status register exception byte if the NAN condition code bit is set when a conditional instruction is executed.

Mnemonic	Definition	Equation	Predicate
EQ	Equal	Z	000001
NE	Not Equal	$\bar{Z}$	001110
GT	Greater Than	$\overline{NANvZvN}$	010010
NGT	Not Greater Than	$NANvZvN$	011101
GE	Greater Than or Equal	$Zv(\overline{NANvN})$	010011
NGE	Not (Greater Than or Equal)	$NANv(N\bar{A}\bar{Z})$	011100
LT	Less Than	$N\bar{A}(\overline{NANvZ})$	010100
NLT	Not Less Than	$NANv(Zv\bar{N})$	011011
LE	Less Than or Equal	$Zv(N\bar{A}\bar{NAN})$	010101
NLE	Not (Less Than or Equal)	$NANv(\bar{N}vZ)$	011010
GL	Greater or Less Than	$\overline{NANvZ}$	010110
NGL	Not (Greater or Less Than)	$NANvZ$	011001
GLE	Greater, Less or Equal	$\overline{NAN}$	010111
NGLE	Not (Greater, Less or Equal)	NAN	011000

where:

"v" = Logical OR

"A" = Logical AND

The following conditional tests do not set the BSUN bit in the status register exception byte under any circumstances.

Mnemonic	Definition	Equation	Predicate
EQ	Equal	Z	000001
NE	Not Equal	$\bar{Z}$	001110
OGT	Ordered Greater Than	$\overline{N \wedge vZ \wedge N}$	000010
ULE	Unordered or Less or Equal	$N \wedge vZ \wedge N$	001101
OGE	Ordered Greater Than or Equal	$Z \vee (\overline{N \wedge v \bar{N}})$	000011
ULT	Unordered or Less Than	$N \wedge v(N \wedge \bar{Z})$	001101
OLT	Ordered Less Than	$N \wedge (\overline{N \wedge vZ})$	000100
UGE	Unordered or Greater or Equal	$N \wedge vZ \wedge N$	001011
OLE	Ordered Less Than or Equal	$Z \vee (N \wedge \overline{N \wedge N})$	000101
UGT	Unordered or Greter Than	$N \wedge v(\bar{N} \vee Z)$	001010
OGL	Ordered Greater or Less Than	$\overline{N \wedge vZ}$	000110
UEQ	Unordered or Equal	$N \wedge vZ$	001001
OR	Ordered	$\overline{N \wedge N}$	000111
UN	Unordered	$N \wedge N$	001000

where:

"v" = Logical OR

"^" = Logical AND

The following tests are not generally used but are implemented for completeness of the set. If the NAN condition code bit is set, T and F do not set the BSUN bit, but SF, ST, SEQ, and SNE do set the BSUN bit.

Mnemonic	Definition	Equation	Predicate
F	False	False	000000
T	True	True	001111
SF	Signaling False	False	010000
ST	Signaling True	True	011111
SEQ	Signaling Equal	Z	010001
SNE	Signaling Not Equal	$\bar{Z}$	011110

4.5 DETAILED INSTRUCTION DESCRIPTIONS

Subsequent paragraphs contain detailed information about each instruction in the FPCP instruction set. Instructions are arranged in alphabetical order by assembler mnemonic. The following paragraphs provide background information to aid in reading the detailed instruction information presented.

4.5.1 Addressing Modes

Due to the nature of the MC68020/MC68030 and FPCP coprocessor interface, the FPCP supports all MC68020/MC68030 addressing modes. The MC68020/MC68030 effective address modes are categorized by the manner in which the modes are used. The following classifications are used in the instruction details.

Data	If an effective address is used to refer to data operands, it is considered a data addressing mode.
Memory	If an effective address is used to refer to memory operands, it is considered a memory addressing mode.
Alterable	If an effective address is used to refer to alterable (writable) operands, it is considered an alterable addressing mode.
Control	If an effective address is used to refer to memory operands that do not have an associated size, it is considered a control addressing mode.

Table 4-10 shows the various addressing categories of each effective address mode. These categories may be combined so that additional, more restrictive, classifications may be defined. For example, the instruction descriptions use such classifications as memory alterable or data alterable. The former refers to those addressing modes which are both memory and alterable addresses (i.e., the intersection of the two sets of modes), and the latter refers to addressing modes which are both data and alterable.

Table 4-10. Effective Addressing Mode Categories

Address Modes	Mode	Register	Data	Memory	Control	Alterable	Assembler Syntax
Data Register Direct	000	reg. no.	X	—	—	X	Dn
Address Register Direct	001	reg. no.	—	—	—	X	An
Address Register Indirect	010	reg. no.	X	X	X	X	(An)
Address Register Indirect with Postincrement	011	reg. no.	X	X	—	X	(An) +
Address Register Indirect with Predecrement	100	reg. no.	X	X	—	X	– (An)
Address Register Indirect with Displacement	101	reg. no.	X	X	X	X	(d ₁₆ ,An)
Address Register Indirect with Index (8-Bit Displacement)	110	reg. no.	X	X	X	X	(dg,An,Xn)
Address Register Indirect with Index (Base Displacement)	110	reg. no.	X	X	X	X	(bd,An,Xn)
Memory Indirect Postindexed	110	reg. no.	X	X	X	X	{(bd,An),Xn,od}
Memory Indirect Preindexed	110	reg. no.	X	X	X	X	{(bd,An,Xn),od}
Absolute Short	111	000	X	X	X	X	(xxx).W
Absolute Long	111	001	X	X	X	X	(xxx).L
Program Counter Indirect with Displacement	111	010	X	X	X	—	(d ₁₆ ,PC)
Program Counter Indirect with Index (8-Bit Displacement)	111	011	X	X	X	—	(dg,PC,Xn)
Program Counter Indirect with Index (Base Displacement)	111	011	X	X	X	—	(bd,PC,Xn)
PC Memory Indirect Postindexed	111	011	X	X	X	—	{(bd,PC),Xn,od}
PC Memory Indirect Preindexed	111	011	X	X	X	—	{(bd,PC,Xn),od}
Immediate	111	100	X	X	—	—	#(data)

4.5.2 Instruction Description Format

The details of each instruction are provided in **4.6 INDIVIDUAL INSTRUCTION DESCRIPTIONS**. Figure 4-1 illustrates what information is given in these instruction descriptions.

4.5.3 Operation Tables

An operation table is included for most instructions. This table lists the result data types for the instruction based on types of input operand(s). For example, Figure 4-2 illustrates the table for the FADD instruction.

In this table, the type of source operand is shown along the top, and the type of the destination operand is shown along the side. In-range numbers are normalized, denormalized, or unnormalized real numbers, integers, or packed decimal numbers that are converted to normalized or denormalized extended precision numbers upon entering the FPCP.

From this table, it can be seen that if both the source and destination operand are positive zero, the result is also a positive zero. For another example, if the source operand is a positive zero and the destination operand is an in-range number, then the ADD algorithm

INSTRUCTION NAME

FABS

OPERATION DESCRIPTION (SEE 4.6 INDIVIDUAL INSTRUCTION DESCRIPTIONS FOR NOTATION DEFINITIONS).

Operation: Absolute Value of Source

SYNTAX FOR THIS INSTRUCTION

Assembler Syntax: FABS.<fmt> <ea>
FABS.X FPrr
FABS.X FPN

Attributes: Format = (Byte, Word,

TEXT DESCRIPTION OF INSTRUCTION OPERATION

Description: Converts the source operand to the absolute value of that number in

RESULT OF OPERATION FOR INPUT OPERAND(S). (THIS TABLE DEFINES THE DATA TYPE OF THE RESULT THAT IS RETURNED FOR EACH COMBINATION OF INPUT OPERANDS.)

Operation Table:

Destination \ Source	Int	FP
Result	Int	FP

NOTE: If the source operand is a

STATUS REGISTER EFFECTS

Status Register:

Condition Codes: Affected

Quotient Byte: Not affected

Exception Byte: BSUN
SNAN
OPERR
OVFL
UNFL

DZ
INEX2
INEX1

INSTRUCTION FORMAT (THIS SPECIFIES THE BIT PATTERN AND FIELDS OF THE OPERATION AND COMMAND WORDS, AND ANY OTHER WORDS THAT ARE ALWAYS PART OF THE INSTRUCTION. THE EFFECTIVE ADDRESS EXTENSIONS ARE NOT EXPLICITLY ILLUSTRATED. THE EXTENSION WORDS (IF ANY) FOLLOW IMMEDIATELY AFTER THE ILLUSTRATED PORTIONS OF THE INSTRUCTIONS. REFER TO THE USER'S MANUAL OF THE MC68020 OR MC68030 FOR THE FORMAT OF ANY REQUIRED EXTENSION WORDS.)

Accrued Exception Byte: Affected

Instruction Format:

MEANINGS AND ALLOWED VALUES (FOR THE VARIOUS FIELDS REQUIRED BY THE INSTRUCTION (FORMAT)).

15	14	13	12	11	10
1	1	1	1	COPROCESSOR ID	
0	R/M	0	SOURCE SPECIFIER		

Figure 4-1. Instruction Description Format

Source Destination	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Add	Add	+inf	-inf
Zero	+	-	Add	+0.0 0.0 ¹	+inf	-inf
Zero	+	-	+inf -inf	+inf -inf	+inf NAN ²	NAN ² -inf

NOTES:

1. Returns +0.0 in rounding modes RN, RZ, and RP; returns -0.0 in RM.
2. Sets the OPERR bit in the FPSR exception byte.
3. If either operand is a NAN, refer to 4.5.4 NANs for more information.

Figure 4-2. Operation Table Example (FADD Instruction)

is executed to obtain the result. If a label such as ADD appears in the table, it indicates that the FPCP performs the indicated operation and returns the correct result.

A third example of using the tables is when a source operand is plus infinity and the destination operand is minus infinity. Since the result of such an operation is undefined, a not-a-number (NAN) is returned as the result, and the OPERR bit is set in the FPSR exception byte.

4.5.4 NANs

In addition to the data types covered in the operation tables for each instruction, NANs can also be used as inputs to an arithmetic operation. The operation tables do not contain a row and column for NANs because NANs are handled the same way in all operations.

4.5.4.1 NONSIGNALING NANs. If either, but not both, operand of an operation is a NAN, and it is a nonsignaling NAN, then that NAN is returned as the result. If both operands are nonsignaling NANs, then the destination operand nonsignaling NAN is returned as the result.

4.5.4.2 SIGNALING NANs. If either operand to an operation is a signaling NAN (SNAN), then the SNAN bit is set in the FPSR EXC byte. If the SNAN trap enable bit is set in the FPCR ENABLE byte, then the trap is taken and the destination is not modified. If the SNAN trap enable bit is not set, then the SNAN is converted to a nonsignaling NAN (by setting the SNAN bit in the operand to a one), and the operation continues as described in the preceding paragraph for nonsignaling NANs.

4.5.5 Operation Post Processing

Most floating-point operations end with an identical post processing step. While reading the summary for each instruction, it should be assumed that an instruction performs post processing unless the summary specifically states that the instruction does not do so. The following paragraphs describe post processing in detail.

4.1 SETTING FLOATING-POINT CONDITION CODES. Unlike the integer arithmetic condition codes found in the MC68020/MC68030, which are set uniquely for each instruction, the floating-point condition codes are either not changed by an instruction or are always set in the same way by an instruction. Therefore, it is not necessary to include details of condition code settings for each FPCP instruction in the detailed instruction descriptions. The following paragraphs describe how condition codes are set for all instructions that modify any condition codes.

Refer to **2.3.1 FPSR Floating-Point Condition Code Byte** for a description of the FPSR condition code byte. The four condition code bits are:

N	Sign of Mantissa	I	Infinity
Z	Zero	NAN	Not-A-Number

These condition code bits differ slightly from integer condition codes. The floating-point condition codes are not dependent on the type of operation being performed, but rather, can be set at the end of the operation by examining the result. (The M68000 integer condition code bits N and Z have this characteristic, but the V and C bits are set differently for different instructions.) At the end of any floating-point operation, the result is inspected, and the condition code bits are set or cleared accordingly. For example, if the result of an operation is a positive normalized number, then all of the condition code bits are set to zero. If the result is a minus infinity, then the N and I bits are set, and the Z and NAN bits are cleared.

Refer to **2.3.1 FPSR Floating-Point Condition Code Byte** for a description of the use of these bits to generate the four conditions required by the IEEE floating-point standard. Refer to **4.4 CONDITIONAL TEST DEFINITIONS** for a description of the use of the four condition code bits to generate the 32 floating-point conditional tests.

4.5.5.2 UNDERFLOW, ROUND, OVERFLOW. During calculation of an arithmetic result, the ALU of the FPCP has more precision and range than the 80-bit extended precision format. However, the final result of these operations is an extended precision floating-point value. In some cases, an internal result becomes either smaller or larger than can be represented in extended precision. Also, the operation may have generated a larger exponent or more bits of precision than can be represented in the chosen rounding precision. For these reasons, every arithmetic instruction ends by rounding the result and checking for overflow and underflow.

At the completion of an arithmetic operation, the internal result is checked to see if it is too small to be represented as a normalized number in the selected precision. If so, the underflow (UNFL) bit is set in the FPSR EXC byte. It is also denormalized unless denormalization provides a zero value. Denormalizing a number causes a loss of accuracy, but a zero is not returned unless absolutely necessary. If a number is grossly underflowed, the FPCP returns a correctly signed zero or the correctly signed smallest denormalized number, depending on the rounding mode in effect. For more details on underflow, refer to **6.1.5 Underflow**.

If no underflow occurs, the internal result is rounded according to the user-selected rounding precision and rounding mode. Refer to Figure 6-3 for a detailed description of rounding. After rounding, the inexact bit (INEX2) is set appropriately. Lastly, the magnitude of the result is checked to see if it is too large to be represented in the current rounding precision. If so, the overflow (OVFL) bit is set and a correctly signed infinity or correctly signed largest

Two important exceptions to the above description are: the execution of the FSGLDIV instruction and of the FSGLMUL instruction. For these two instructions, the rounding precision programmed in the mode control byte is ignored (although the selected rounding mode is used). The input operands to these instructions are assumed to be single-precision values, but no checking is performed to verify the inputs (each mantissa is truncated to 23 bits, and the exponent is accepted as an extended-precision value).

These two instructions first check the intermediate result for underflow as previously described, but use the underflow threshold of extended precision regardless of the selected rounding precision. If no underflow occurs, the mantissa is rounded to the single-precision boundary and is denormalized if necessary. Finally, the exponent is checked for overflow, again using the overflow threshold of extended precision. Thus, the final result generated has the range of an extended-precision number with a mantissa accurate to only 23 bits. If an underflow or overflow occurs, the correctly signed number returned (largest normalized number, infinity, zero, or smallest denormalized number) is an extended precision-number with an extended-precision mantissa value.

4.6 INDIVIDUAL INSTRUCTION DESCRIPTIONS

The following notation is used in the detailed instruction definitions that follow:

(operand)	Contents of the referenced location or register.
<fmt>	Operand data format: Byte, word, long, single, double, extended, or packed (denoted in the assembler syntax as an extension to the instruction mnemonic of .B, .W, .L, .S, .D, .X, or .P, respectively).
<ea>	Any valid MC68020/MC68030 addressing mode.
<label>	A relative label used by an assembler to calculate a displacement.
<list>	A list of the floating-point data registers or control registers.
♦	The left operand is moved to the location specified by the right operand.
FPcr	One of the three floating-point system control registers (FPCR, FPSR, or FPIAR).
FPn	One of eight floating-point data registers (always specifies the destination register).
FPm	One of eight floating-point data registers (always specifies the source register).
FPc:FPs	Two of eight floating-point data registers. This notation is used only with the FSINCOS instruction and specifies the register pair where the cosine and sine values are stored.
+inf	Positive infinity
-inf	Negative infinity
NAN	Not-A-Number
d	Displacement
k	An integer (–64 to +17) that specifies the format of a number to be stored in the packed BCD format.
ccc	An index into the FPCP constant ROM.

Operation: Absolute Value of Source $\rightarrow$ FPN

Assembler FABS.<fmt> <ea>,FPn

Syntax: FABS.X FPM,FPn
FABS.X FPN

4

Attributes: Format = {Byte, Word, Long, Single, Double, Extended, Packed}

Description: Converts the source operand to extended precision (if necessary) and stores the absolute value of that number in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Absolute Value		Absolute Value		Absolute Value	

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES.

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs
OPERR	Cleared
OVFL	Cleared
UNFL	If the source is an extended precision denormalized number, refer to 6.1.5 Underflow; cleared otherwise.
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
{d16,An}	101	reg. number:An	{d16,PC}	111	010
{d8,An,Xn}	110	reg. number:An	{d8,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is then written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Arc Cosine of Source $\rightarrow$ FPn

Assembler FACOS.<fmt> <ea>,FPn

Syntax: FACOS.X FPm,FPn
FACOS.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the arc cosine of that number. Stores the result in the destination floating-point data register. This function is not defined for source operands outside of the range $[-1 \dots +1]$; if the source is not in the correct range, a NAN is returned as the result and the OPERR bit is set in the FPSR. If the source is in the correct range, the result is in the range of $[0 \dots \pi]$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Arc Cosine		$-\pi/2$		NAN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES.

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source is infinity, > -1 or < -1 ; cleared otherwise.
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	C	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
((bd,An,Xn),od)	110	reg. number:An	((bd,PC,Xn),od)	111	011
((bd,An),Xn,od)	110	reg. number:An	((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is then written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Source + FPn $\rightarrow$ FPn

Assembler FADD.<fmt> <ea>,FPn

Syntax: FADD.X FPm,FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and adds that number to the number contained in the destination floating-point data register. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Add		+ inf	- inf
Zero	+	-	Add		+ 0.0 0.0 ¹	- 0.0 ¹
Infinity	+	-	+ inf	- inf	- inf	NAN ²

NOTES:

1. Returns +0.0 in rounding modes RN, RZ, and RP; returns -0.0 in RM.
2. Sets the OPERR bit in the FPSR exception byte.
3. If either operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES.

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source and the destination are opposite-signed infinities; cleared otherwise.
OVFL	Refer to 6.1.4 Overflow.
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID		0	0	0	EFFECTIVE ADDRESS MODE						REGISTER
0	R/M	0	SOURCE SPECIFIER		DESTINATION REGISTER		0	1	0	0	0	0	1	0	

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
{d ₁₆ ,An}	101	reg. number:An	{d ₁₆ ,PC}	111	010
{d ₈ ,An,Xn}	110	reg. number:An	{d ₈ ,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: Arc Sine of the Source $\rightarrow$ FPn

Assembler FASIN.<fmt> <ea>,FPn

Syntax: FASIN.X FPm,FPn

FASIN.X FPn

4

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the arc sine of the number. Stores the result in the destination floating-point data register. This function is not defined for source operands outside of the range $[-1 \dots +1]$; if the source is not in the correct range, a NAN is returned as the result and the OPERR bit is set in the FPSR. If the source is in the correct range, the result is in the range of $[-\pi/2 \dots +\pi/2]$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Arc Sine		+0.0	-0.0	NAN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES.

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source is infinity, $> +1$ or < -1 ; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
((bd,An,Xn),od)	110	reg. number:An	((bd,PC,Xn),od)	111	011
((bd,An),Xn,od)	110	reg. number:An	((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is then written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

4

Operation: Arc Tangent of Source $\rightarrow$ FPn

Assembler FATAN.<fmt> <ea>,FPn

Syntax: FATAN.X FPm,FPn
FATAN.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the arc tangent of that number. Stores the result in the destination floating-point data register. The result is in the range of $[-\pi/2 \dots +\pi/2]$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Arc Tangent		+ 0.0	- 0.0	$-\pi/2$	$+\pi/2$

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES.

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Cleared
OVFL	Cleared
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS					
				SOURCE SPECIFIER			DESTINATION REGISTER			MODE	REGISTER				
0	R/M	0								0	0	0	1	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is then written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Hyperbolic Arc Tangent of Source $\rightarrow$ FPn

Assembler FATANH.<fmt> <ea>,FPn

Syntax: FATANH.X FPm,FPn
FATANH.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the hyperbolic arc tangent of that value. Stores the result in the destination floating-point data register. This function is not defined for source operands outside of the range $(-1 \dots +1)$; and the result is equal to $-\infty$ or $+\infty$ if the source is equal to $+1$ or -1 , respectively. If the source is outside of the range $[-1 \dots +1]$, a NAN is returned as the result and the OPERR bit is set in the FPSR.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Hyperbolic Arc Tangent		+0.0	-0.0	NAN ¹	

NOTE:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source is $> +1$ or < -1 ; cleared otherwise
OVFL	Cleared
UNFL	Refer to 6.1.5 Underflow.
DZ	Set if the source is equal to ± 1 or -1 ; cleared otherwise
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(d8,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d16,PC)	111	010
(d8,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is then written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: If condition true, then $PC + d \rightarrow PC$

Assembler Syntax: FBcc.<size> <label>

Attributes: Size = (Word, Long)

4

Description: If the specified floating-point condition is met, program execution continues at the location (PC) + displacement. The displacement is a two's-complement integer that counts the relative distance in bytes. The value of the PC used to calculate the destination address is the address of the branch instruction plus two. If the displacement size is word, then a 16-bit displacement is stored in the word immediately following the instruction operation word. If the displacement size is long word, then a 32-bit displacement is stored in the two words immediately following the instruction operation word.

The conditional specifier cc selects any one of the 32 floating-point conditional tests as described in **4.4 CONDITIONAL TEST DEFINITIONS**.

Status Register:

Condition Codes: Not affected

Quotient Byte: Not affected

Exception Byte: BSUN Set if the NAN condition code is set and the condition selected is an IEEE nonaware test

SNAN Not Affected

OPERR Not Affected

OVFL Not Affected

UNFL Not Affected

DZ Not Affected

INEX2 Not Affected

INEX1 Not Affected

Accrued Exception Byte: The IOP bit is set if the BSUN bit is set in the exception byte. No other bit is affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	1	SIZE	CONDITIONAL PREDICATE					
16-BIT DISPLACEMENT, OR MOST SIGNIFICANT WORD OF 32-BIT DISPLACEMENT															
LEAST SIGNIFICANT WORD OF 32-BIT DISPLACEMENT (IF NEEDED)															

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Size Field — Specifies the size of the signed displacement:

If Format=0, then the displacement is 16-bits and is sign extended before use.

If Format=1, then the displacement is 32-bits.

Conditional Predicate Field — Specifies one of 32 conditional tests as defined in **4.4 CONDITIONAL TEST DEFINITIONS**.

NOTE

When a BSUN exception occurs, the main processor takes a pre-instruction exception. If the exception handler returns without modifying the image of the PC on the stack frame (to point to the instruction following the FBcc), then it must clear the cause of the exception (by clearing the NAN bit or disabling the BSUN trap) or the exception occurs again immediately upon return to the routine that caused the exception.

Operation: FPn — Source

Assembler FCMP.<fmt> <ea>,FPn

Syntax: FCMP.X FPm,FPn

Attributes: Format = {Byte, Word, Long, Single, Double, Extended, Packed}

4

Description: Converts the source operand to extended precision (if necessary) and subtracts the operand from the destination floating-point data register. The result of the subtraction is not retained, but it is used to set the floating-point condition codes as described in **4.5.5.1 SETTING FLOATING-POINT CONDITION CODES**.

Operation Table The entries in this operation table differ from those of the tables describing most of the FPCP instructions. For each combination of input operand types, the condition code bits that may be set are indicated. If the name of a condition code bit is given and is not enclosed in brackets, then it is always set. If the name of a condition code bit is enclosed in brackets, then that bit is either set or cleared, as appropriate. If the name of a condition code bit is not given, then that bit is always cleared by the operation. The infinity bit is always cleared by the FCMP instruction, since it is not used by any of the conditional predicate equations. Note that the NAN bit is not shown, since NANs are always handled in the same manner (as described in **4.5.4 NANs**).

Destination \ Source		In Range		Zero		Infinity	
		+	-	+	-	+	-
In Range	+	{NZ}	none	none	none	N	none
	-	N	{NZ}	N	N	N	none
Zero	+	N	none	Z	Z	N	none
	-	N	none	NZ	NZ	N	none
Infinity	+	none	none	none	none	Z	none
	-	N	N	N	N	N	NZ

NOTE: If either operand is a NAN, refer to **4.5.4 NANs** for more information.

Status Register:

Condition Codes: Affected as described in the operation table above

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs .
OPERR	Cleared
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input ; cleared otherwise

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	1	1	0	0	0

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(dg,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn,od)	110	reg. number:An
([bd,An,Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d ₁₆ ,PC)	111	010
(dg,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn,od)	111	011
([bd,PC,Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: Cosine of Source $\rightarrow$ FPN

Assembler FCOS.<fmt> <ea>,FPn

Syntax: FCOS.X FPN,FPn
FCOS.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the cosine of that number. Stores the result in the destination floating-point data register. This function is not defined for source operands of $(\pm)$ infinity. If the source operand is not in the range of $[-2\pi \dots +2\pi]$, then the argument is reduced to within that range before the cosine is calculated. However, large arguments may lose accuracy during reduction, and very large arguments (greater than approximately 10^{20}) lose all accuracy. The result is in the range of $[-1 \dots +1]$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Cosine		+1.0		NaN ¹	

NOTE:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source operand is $(+/-)$ infinity; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should contain zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Hyperbolic Cosine of Source $\rightarrow$ FPN

Assembler: FCOSH.<fmt> <ea>,FPn

Syntax: FCOSH.X FPM,FPn
FCOSH.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the hyperbolic cosine of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Hyperbolic Cosine		+ 1.0		+ inf	

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Cleared
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M = 0, this field is unused, and should be all zeros.

If R/M = 1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M = 0, specifies the source floating-point data register, FPM.

If R/M = 1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M = 0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: If condition true then no operation
 else $Dn - 1 \nrightarrow Dn$
 if $Dn \neq -1$
 then $PC + d \nrightarrow PC$
 else execute next instruction

Assembler Syntax: FDBcc $Dn, <label>$

Attributes: Unsized

Description: This instruction is a looping primitive of three parameters: a floating-point condition, a counter (an MPU data register) and a 16-bit displacement. The FPCP first tests the condition to determine if the termination condition for the loop has been met, and if so, the main processor proceeds to execute the next instruction in the instruction stream. If the termination condition is not true, the low order 16-bits of the counter register are decremented by one. If the result is -1 , the count is exhausted, and execution continues with the next instruction. If the result is not equal to -1 , execution continues at the location specified by the current value of the PC plus the sign-extended 16-bit displacement. The value of the PC used in the branch address calculation is the address of the displacement word.

The conditional specifier cc selects any one of the 32 floating-point conditional tests as described in **4.4 CONDITIONAL TEST DEFINITIONS**.

Status Register:

Condition Codes:	Not affected	
Quotient Byte:	Not affected	
Exception Byte:	BSUN	Set if the NAN condition code is set and the condition selected is an IEEE nonaware test
	SNAN	Not Affected
	OPERR	Not Affected
	OVFL	Not Affected
	UNFL	Not Affected
	DZ	Not Affected
	INEX2	Not Affected
	INEX1	Not Affected

Accrued Exception Byte: The IOP bit is set if the BSUN bit is set in the exception byte. No other bit is affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	0	0	1	COUNT REGISTER		
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					
16-BIT DISPLACEMENT															

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. **Freescale** assemblers default to ID=1 for the FPCP.

Count Register Field — Specifies main processor data register that is used as the counter.

Conditional Predicate Field — Specifies one of the 32 floating-point conditional tests as described in **4.4 CONDITIONAL TEST DEFINITIONS**.

Displacement Field — Specifies the branch distance (from the address of the instruction plus two) to the destination in bytes.

NOTES:

1. The terminating condition is like that defined by the UNTIL loop constructs of high-level languages. For example: FDBOLT can be stated as "decrement and branch until ordered less than".
2. There are two basic ways of entering a loop: at the beginning, or by branching to the trailing FDBcc instruction. If a loop structure terminated with FDBcc is entered at the beginning, the control counter must be one less than the number of loop executions desired. This count is useful for indexed addressing modes and dynamically specified bit operations. However, when entering a loop by branching directly to the trailing FDBcc instruction, the count should equal the loop execution count. In this case, if the counter is zero when the loop is entered, the FDBcc instruction does not branch, causing a complete bypass of the main loop.
3. When a BSUN exception occurs, a pre-instruction exception is taken by the main processor. If the exception handler returns without modifying the image of the PC on the stack frame (to point to the instruction following the FDBcc), then it must clear the cause of the exception (by clearing the NAN bit or disabling the BSUN trap) or the exception occurs again immediately upon return to the routine that caused the exception.

Operation: $FPn (\div) \text{Source} \rightarrow FPn$

Assembler FDIV.<fmt> <ea>,FPn

Syntax: FDIV.X FPm,FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and divides that number into the number in the destination floating-point data register. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Divide	Divide	Divide	Divide
Zero	+	-	+0.0	-0.0	NAN ²	NAN ²
Infinity	+	-	+inf	-inf	NAN ²	NAN ²

NOTES:

1. Sets the DZ bit in the FPSR exception byte.
2. Sets the OPERR bit in the FPSR exception byte.
3. If either operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes:	Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES	
Quotient Byte:	Not affected	
Exception Byte:	BSUN	Cleared
	SNAN	Refer to 4.5.4 NANs.
	OPERR	Set for $0(\div)0$ or $\text{infinity}(\div)\text{infinity}$; cleared otherwise
	OVFL	Refer to 6.1.4 Overflow.
	UNFL	Refer to 6.1.5 Underflow.
	DZ	Set if the source is zero and the destination is in range; cleared otherwise
	INEX2	Refer to 6.1.7 Inexact Result.
	INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M = 0, this field is unused, and should be all zeros.

If R/M = 1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number:An	# data	111	100
{An} +	011	reg. number:An			
~ {An}	100	reg. number:An			
{d ₁₆ ,An}	101	reg. number:An	{d ₁₆ ,PC}	111	010
{dg,An,Xn}	110	reg. number:An	{dg,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M = 0, specifies the source floating-point data register, FPM.

If R/M = 1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

Operation: e(Source) $\rightarrow$ FPn

Assembler FETOX.<fmt> <ea>,FPn

Syntax: FETOX.X FPm,FPn
FETOX.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates e to the power of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	e ^x		+ 1.0		- inf	- 0.0

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. **Freesc**ale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, **Freesc**ale assemblers set the source and destination fields to the same value.

Operation: $e(\text{Source}) - 1 \neq \text{FPn}$

Assembler FETOXM1.<fmt> <ea>,FPn

Syntax: FETOXM1.X FPm,FPn
FETOXM1.X FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates e to the power of that number. Then, subtracts one from that value, and stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	$e^x - 1$		+0.0	-0.0	-inf	-1.0

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	C
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	0	0	C

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
{An}	010	reg. number:An
{An}+	011	reg. number:An
-(An)	100	reg. number:An
{d16,An}	101	reg. number:An
{dg,An,Xn}	110	reg. number:An
{bd,An,Xn}	110	reg. number:An
{[bd,An,Xn],od}	110	reg. number:An
{[bd,An],Xn,od}	110	reg. number:An

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	111	100
{d16,PC}	111	010
{dg,PC,Xn}	111	011
{bd,PC,Xn}	111	011
{[bd,PC,Xn],od}	111	011
{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Exponent of Source $\rightarrow$ FPn

Assembler FGETEXP.<fmt> <ea>,FPn

Syntax: FGETEXP.X FPm,FPn
FGETEXP.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and extracts the binary exponent. Removes the exponent bias, converts the exponent to an extended precision floating-point number, and stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Exponent		+0.0	-0.0	NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source is (+ or -)infinity; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
((bd,An,Xn),od)	110	reg. number:An	((bd,PC,Xn),od)	111	011
((bd,An),Xn,od)	110	reg. number:An	((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Mantissa of Source $\rightarrow$ FPN

Assembler: FGETMAN.<fmt> <ea>,FPn

Syntax: FGETMAN.X FPM,FPn
FGETMAN.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and extracts the mantissa. Converts the mantissa to an extended precision value and stores the result in the destination floating-point data register. The result is in the range [1.0 . . . 2.0) with the sign of the source mantissa, zero, or is a NAN.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Mantissa		+ 0.0	- 0.0	NAN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source is (+ or -)infinity; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	1	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Integer Part of Source $\rightarrow$ FPn

Assembler Syntax: FINT.<fmt> <ea>,FPn
FINT.X FPm,FPn
FINT.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and extracts the integer part and converts it to an extended precision floating-point number. Stores the result in the destination floating-point data register. The integer part is extracted by rounding the extended precision number to an integer using the current rounding mode selected in the FPCR mode control byte. Thus, the integer part returned is the number that is to the left of the radix point when the exponent is zero, after rounding. For example, the integer part of 137.57 is 137.0 for the round-to-zero and round-to-minus infinity modes, and 138.0 for the round-to-nearest and round-to-plus infinity modes. Note that the result of this operation is a floating-point number.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Integer		+0.0	-0.0	+inf	-inf

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Cleared
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
{(bd,An,Xn),od}	110	reg. number:An	{(bd,PC,Xn),od}	111	011
{(bd,An,Xn,od)}	110	reg. number:An	{(bd,PC,Xn,od)}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Integer Part of Source $\rightarrow$ FPN

Assembler FINTRZ.<fmt> <ea>,FPn

Syntax: FINTRZ.X FPM,FPn
FINTRZ.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and extracts the integer part and converts it to an extended precision floating-point number. Stores the result in the destination floating-point data register. The integer part is extracted by rounding the extended precision number to an integer using the round-to-zero mode, regardless of the rounding mode selected in the FPCR mode control byte (making it useful for FORTRAN assignments). Thus, the integer part returned is the number that is to the left of the radix point when the exponent is zero. For example, the integer part of 137.57 is 137.0; the integer part of 0.1245×10^2 is 12.0. Note that the result of this operation is a floating-point number.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Integer, Forced Round-To-Zero		+0.0	-0.0	+inf	-inf

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Cleared
UNFL Cleared
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	1	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
-(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, F_P_n.

If RM=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, F_P_n.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Log10 of Source $\rightarrow$ FPn

Assembler: FLOG10.<fmt> <ea>,FPn

Syntax: FLOG10.X FPm,FPn
FLOG10.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Convert the source operand to extended precision (if necessary) and calculates the logarithm of that number using base 10 arithmetic. Stores the result in the destination floating-point data register. This function is not defined for input values less than zero.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Log10	NAN ¹	-inf ²		+inf	NAN ¹

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. Sets the DZ bit in the FPSR exception byte.
3. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source operand is <0; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Set if the source is (+ or -); cleared otherwise
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
((bd,An,Xn),od)	110	reg. number:An	((bd,PC,Xn),od)	111	011
((bd,An),Xn,od)	110	reg. number:An	((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Log2 of Source $\rightarrow$ FPn

Assembler Syntax: FLOG2.<fmt> <ea>,FPn
FLOG2.X FPm,FPn
FLOG2.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the logarithm of that number using base two arithmetic. Stores the result in the destination floating-point data register. This function is not defined for input values less than zero.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Log2	NAN ¹	-inf ²		+inf	NAN ¹

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. Sets the DZ bit in the FPSR exception byte.
3. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source is < 0; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Set if the source is (+ or -)0; cleared otherwise
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Log_e of Source $\rightarrow$ FPn

Assembler Syntax: FLOGN.<fmt> <ea>,FPn
FLOGN.X FPm,FPn
FLOGN.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the natural logarithm of that number. Stores the result in the destination floating-point data register. This function is not defined for input values less than zero.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	ln(x)	NAN ¹	-inf ²		+inf	NAN ¹

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. Sets the DZ bit in the FPSR exception byte.
3. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Set if the source operand is < 0; cleared otherwise
OVFL Cleared
UNFL Cleared
DZ Set if the source is (+ or -)0; cleared otherwise
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
{d16,An}	101	reg. number:An	{d16,PC}	111	010
{dg,An,Xn}	110	reg. number:An	{dg,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Log_e of (Source + 1) $\rightarrow$ FPn

Assembler FLOGNP1.<fmt> <ea>,FPn

Syntax: FLOGNP1.X FPm,FPn
FLOGNP1.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary), adds one to that value, and calculates the natural logarithm of that intermediate result. Stores the result in the destination floating-point data register. This function is not defined for input values less than -1 .

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	$\ln(x+1)$	$\ln(x+1)^1$	+0.0	-0.0	-inf	NAN ²

NOTES:

1. If the source is -1 , sets the DZ bit in the FPSR exception byte and returns a NAN. If the source is < -1 , sets the OPERR bit in the FPSR exception byte and returns a NAN.
2. Sets the OPERR bit in the FPSR exception byte.
3. If the source operand is a NAN, refer to **4.5.4 NANs** for more information.

Status Register:

Condition Codes: Affected as described in **4.5.5.1 SETTING FLOATING-POINT CONDITION CODES**

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs .
OPERR	Set if the source operand is < -1 ; cleared otherwise
OVFL	Cleared
UNFL	Refer to 6.1.5 Underflow .
DZ	Set if the source operand is -1 ; cleared otherwise
INEX2	Refer to 6.1.7 Inexact Result .
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input ; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	1	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
((bd,An,Xn),od)	110	reg. number:An
((bd,An),Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011
(bd,PC,Xn)	111	011
((bd,PC,Xn),od)	111	011
((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Modulo Remainder of (FPn (÷) Source) ÷ FPn

Assembler FMOD.<fmt> <ea>,FPn

Syntax: FMOD.X FPm,FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the modulo remainder of the number in the destination floating-point data register, using the source operand as the modulus. Stores the result in the destination floating-point data register, and stores the sign and seven least significant bits of the quotient in the FPSR quotient byte (the quotient is the result of FPn (÷) Source). The modulo remainder function is defined as:

$$FPn - (Source \times N)$$

where:

$N = \text{INT}(FPn / \text{Source})$ in the round-to-zero mode

The FMOD function is not defined for a source operand equal to zero or for a destination operand equal to infinity. Note that this function is not the same as the FREM instruction, which uses the round-to-nearest mode and thus returns the remainder that is required by the *IEEE Specification for Binary Floating-Point Arithmetic*.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Modulo Remainder		NaN ¹	
Zero	+	-	+0.0 -0.0		NaN ¹	
Infinity	+	-	NaN ¹		NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. Returns the value of FPn before the operation. However, the result is processed by the normal instruction termination procedure to round it as required. Thus, an overflow and/or inexact result may occur if the rounding precision has been changed to a smaller size since the FPn value was loaded.
3. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Loaded with the sign and least significant seven bits of the quotient (FPn (÷) Source). The sign of the quotient is the exclusive OR of the sign bits of the source and destination operands.

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source is zero, or the destination is infinity; cleared otherwise
OVFL	Cleared

UNFL Refer to **6.1.5 Underflow**.
DZ Cleared
INEX2 Refer to **6.1.7 Inexact Result**.
INEX1 If <fmt> is Packed, refer to **6.1.8 Inexact Result on Decimal Input**; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: Source $\rightarrow$ Destination

Assembler Syntax: FMOVE.<fmt> <ea>,FPn
FMOVE.<fmt> FPm,<ea>
FMOVE.P FPm,<ea>{Dn}
FMOVE.P FPm,<ea>{#k}

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Moves the contents of the source operand to the destination operand. Although the primary function of this instruction is data movement, it is also considered an arithmetic instruction since conversions from the source operand format to the destination operand format are performed implicitly during the move operation. Also, the source operand is rounded according to the selected rounding precision and mode.

Unlike the M68000 Family integer data movement instruction, the floating-point move instruction does not support a memory-to-memory format (for such transfers, it is much faster to utilize the M68000 Family integer MOVE instruction to transfer the floating-point data than to use the FMOVE instruction). The FMOVE instruction only supports memory-to-register, register-to-register, and register-to-memory operations (in this context, memory may refer to an MPU data register if the data format is byte, word, long or single). The memory-to-register and register-to-register operations use a command word encoding distinctly different from that used by the register-to-memory operation, and these two operation classes are described separately below.

Memory-to-Register or Register-to-Register Operation:

Converts the source operand to an extended precision floating-point number (if necessary) and stores it in the destination floating-point data register. Depending on the source data format and the rounding precision, some operations may produce an inexact result. In the following table, combinations that can produce an inexact result are marked with a dot (•), but all other combinations produce an exact result.

	Source Format:	B	W	L	S	D	X	P
Rounding Precision:	Single			•		•	•	•
	Double						•	•
	Extended							•

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:	BSUN	Cleared
	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Cleared
	OVFL	Cleared
	UNFL	Refer to 6.1.5 Underflow if the source is an extended precision denormalized number; cleared otherwise.
	DZ	Cleared
	INEX2	Refer to 6.1.7 Inexact Result if <fmt> is L, D or X; cleared otherwise.
	INEX1	Refer to 6.1.8 Inexact Result on Decimal Input if <fmt> is P; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

1	5	1	4	1	3	1	2	1	1	1	0	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID				0	0	0	EFFECTIVE ADDRESS MODE				REGISTER						
0	RIM	0	SOURCE SPECIFIER				DESTINATION REGISTER				0	0	0	0	0	0	0				

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	nn1
(An)	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M = 0, specifies the source floating-point data register, FPM.

If R/M = 1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Register-to-Memory Operation:

Rounds the source operand to the size of the specified destination format and stores it at the destination effective address. If the format of the destination is packed decimal, a third operand is required to specify the format of the resultant string. This operand, called the k-factor, is a 7-bit signed integer (twos complement) and may be specified as an immediate value or in a main processor data register. If a data register contains the k-factor, only the least significant seven bits are used, and the rest of the register is ignored.

Status Register:

Condition Codes:	Not affected	
Quotient Byte:	Not affected	
Exception Byte:	BSUN	Cleared
<fmt> is B, W, or L	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Set if the source operand is infinity, or if the destination size is exceeded after conversion and rounding; cleared otherwise
	OVFL	Cleared
	UNFL	Cleared
	DZ	Cleared
	INEX2	Refer to 6.1.7 Inexact Result .
	INEX1	Cleared

<fmt> is S, D, or X	BSUN	Cleared
	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Cleared
	OVFL	Refer to 6.1.4 Overflow .
	UNFL	Refer to 6.1.5 Underflow .
	DZ	Cleared
	INEX2	Refer to 6.1.7 Inexact Result .
	INEX1	Cleared
<fmt> is P	BSUN	Cleared
	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Set if the k-factor > +17, or the magnitude of the decimal exponent exceeds 3 digits; cleared otherwise
	OVFL	Cleared
	UNFL	Cleared
	DZ	Cleared
	INEX2	Refer to 6.1.7 Inexact Result .
	INEX1	Cleared

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	1	1	DESTINATION FORMAT			SOURCE REGISTER			K-FACTOR (IF REQUIRED)						

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Effective Address Field — Encoded with the M68000 addressing mode for the destination operand as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	—	—
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	{d ₁₆ ,PC}	—	—
(dg,An,Xn)	110	reg. number:An	{dg,PC,Xn}	—	—
(bd,An,Xn)	110	reg. number:An	{bd,PC,Xn}	—	—
{(bd,An,Xn),od}	110	reg. number:An	{(bd,PC,Xn),od}	—	—
{(bd,An),Xn,od}	110	reg. number:An	{(bd,PC),Xn,od}	—	—

*Only if <fmt> is Byte, Word, Long, or Single.

Destination Format Field — Specifies the data format of the destination operand:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P{#k}	Packed Decimal Real with static k-factor
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer
111	P{Dn}	Packed Decimal Real with dynamic k-factor

Source Register Field — Specifies the source floating-point data register, FPM.

k-factor Field — Only used if the destination format is Packed Decimal, to specify the format of the decimal string. For any other destination format, this field should be set to all zeros. For a static k-factor, this field is encoded with a twos-complement integer where the value defines the format as follows:

- 64 to 0 — Indicates the number of significant digit to the right of the decimal point (Fortran “F” format).
- +1 to +17 — Indicates the number of significant digits in the mantissa (Fortran “E” format).
- +18 to +63 — Sets the OPERR bit in the FPSR exception byte, treated as +17.

The format of this field for a dynamic k-factor is:

r r r 0 0 0 0

where:

“rrr” is the number of the main processor data register that contains the k-factor value.

The following table gives several examples of how the k-factor value affects the format of the decimal string that is produced by the FPCP. The format of the string that is generated is independent of the source of the k-factor (static or dynamic).

k-Factor	Source Operand Value	Destination String
-5	+ 12345.678765	+ 1.234567877 E + 4
-3	+ 12345.678765	+ 1.2345679 E + 4
-1	+ 12345.678765	+ 1.23457 E + 4
0	+ 12345.678765	+ 1.2346 E + 4
+1	+ 12345.678765	+ 1. E + 4
+3	+ 12345.678765	+ 1.23 E + 4
+5	+ 12345.678765	+ 1.2346 E + 4

Operation: Source → Destination

Assembler Syntax: FMOVE.L <ea>,FPcr
FMOVE.L FPcr,<ea>

Attributes: Size = (Long)

4

Description: Moves the contents of a floating-point system control register into or out of the FPcr (the control registers are the FPcr, FPSR and FPIAR). The external operand may be in memory or a main processor register. A 32-bit transfer is always performed, even though the system control register may not have 32 implemented bits. Unimplemented bits of a control register are read as zeros and are ignored during writes (but must be zero for compatibility with future devices).

This instruction does not cause pending exceptions (other than protocol violations) to be reported to the main processor. Furthermore, a write to the FPcr exception enable byte or the FPSR exception status byte cannot generate a new exception, regardless of the value written.

Status Register: Changed only if the destination is the FPSR; in which case all bits are modified to reflect the value of the source operand.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS					
				ID						MODE		REGISTER			
1	0	dr	REGISTER SELECT		0	0	0	0	0	0	0	0	0	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPcr.

Effective Address Field — Determines the addressing mode for the operation:

Memory-to-Register

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	000	reg. number: Dn	(xxx).W	111	000
An*	001	reg. number: An	(xxx).L	111	001
(An)	010	reg. number: An	#<data>	111	100
(An) +	011	reg. number: An			
-(An)	100	reg. number: An			
(d ₁₆ ,An)	101	reg. number: An	(d ₁₆ ,PC)	111	010
(dg,An,Xn)	110	reg. number: An	(dg,PC,Xn)	111	011
{bd,An,Xn}	110	reg. number: An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number: An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number: An	{[bd,PC],Xn,od}	111	011

*Only if the source register is the FPIAR.

Register-to-Memory

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	000	reg. number:Dn	(xxx).W	111	000
An*	001	reg. number:An	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	—	—
(An)+	011	reg. number:An			
– (An)	100	reg. number:An	(d16,PC)	—	—
(d16,An)	101	reg. number:An	(d8,PC,Xn)	—	—
(d8,An,Xn)	110	reg. number:An	(b8,PC,Xn)	—	—
(b8,An,Xn)	110	reg. number:An	((b8,PC,Xn),od)	—	—
((b8,An,Xn),od)	110	reg. number:An	((b8,PC,Xn,od)	—	—
((b8,An),Xn,od)	110	reg. number:An			

*Only if the destination register is the FPIAR.

dr Field — Specifies the direction of the data transfer.

0 — Move an external operand to the specified system control register.

1 — Move the specified system control register to an external location.

Register Select Field — Specifies the system control register to be moved:

100 FPCR Floating-point Control Register

010 FPSR Floating-point Status Register

001 FPIAR Floating-point Instruction Address Register

Operation: ROM Constant $\rightarrow$ FPN

Assembler

Syntax: FMOVECR.X #ccc,FPn

Attributes: Format = (Extended)

4

Description: Fetches an extended precision constant from the FPCR on-chip ROM, rounds it to the precision specified in the FPCR mode control byte, and stores it in the destination floating-point data register. The constant is specified by a predefined offset into the constant ROM. The values of the constants contained in the ROM are shown in the offset table at the end of this description.

Status Register:

Condition Codes: Affected as described in **4.5.5.1 SETTING FLOATING-POINT CONDITION CODES**

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Cleared
OPERR	Cleared
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result .
INEX1	Cleared

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	0	0	0	0	0	0
0	1	0	1	1	1	DESTINATION REGISTER				ROM OFFSET					

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Destination Register Field — Specifies the destination floating-point data register, FPN.

ROM Offset Field — Specifies the offset into the FPCP on-chip constant ROM where the desired constant is located. The offsets for the available constants are:

Offset	Constant
\$00	π
\$0B	$\text{Log}_{10}(2)$
\$0C	e
\$0D	$\text{Log}_2(e)$
\$0E	$\text{Log}_{10}(e)$
\$0F	0.0
\$30	$1_n(2)$
\$31	$1_n(10)$
\$32	10^0
\$33	10^1
\$34	10^2
\$35	10^4
\$36	10^8
\$37	10^{16}
\$38	10^{32}
\$39	10^{64}
\$3A	10^{128}
\$3B	10^{256}
\$3C	10^{512}
\$3D	10^{1024}
\$3E	10^{2048}
\$3F	10^{4096}

The on-chip ROM contains other constants useful only to the on-chip microcode routines. The values contained at offsets other than those defined above are reserved for the use of Freescale, and may be different on various mask sets of the FPCP.

Operation: Register List ∇ Destination
Source ∇ Register List

Assembler Syntax: FMOVEM.X <list>, <ea>
FMOVEM.X Dn, <ea>
FMOVEM.X <ea>, <list>
FMOVEM.X <ea>, Dn

<list> A list of any combination of the eight floating-point data registers, with individual register names separated by a slash (/); and/or contiguous blocks of registers specified by the first and last register names separated by a dash (-).

Attributes: Format = (Extended)

Description: Moves one or more extended precision numbers to or from a list of floating-point data registers. No conversion or rounding is performed during this operation, and the FPSR is not affected by the instruction. This instruction does not cause pending exceptions (other than protocol violations) to be reported to the main processor.

Any combination of the eight floating-point data registers can be transferred, with the selected registers specified by a user-supplied mask. This mask is an 8-bit number, where each bit corresponds to one register; if a bit is set in the mask, that register is moved. The register select mask may be specified as a static value contained in the instruction, or a dynamic value in the least significant 8-bits of a main processor data register (the remaining bits of the register are ignored).

FMOVEM allows three types of addressing modes: the control modes, the predecrement mode, or the postincrement mode. If the effective address is one of the control addressing modes, the registers are transferred between the FPCP and memory starting at the specified address and up through higher addresses. The order of the transfer is from FP7-FP0.

If the effective address is the predecrement mode, only a register-to-memory operation is allowed. The registers are stored starting at the address contained in the address register and down through lower addresses. Before each register is stored, the address register is decremented by 12 (the size of an extended precision number in memory) and the floating-point data register is then stored at the resultant address. When the operation is complete, the address register points to the image of the last floating-point data register stored. Each register is stored in the format described in **SECTION 3 OPERAND DATA FORMATS**, with the most significant byte of the register image stored at the lowest address, and the least significant byte at the highest address. The order of the transfer is from FP7-FP0.

If the effective address is the postincrement mode, only a memory-to-register operation is allowed. The registers are loaded starting at the specified address and up

through higher addresses. After each register is stored, the address register is incremented by 12 (the size of an extended precision number in memory). When the operation is complete, the address register points to the byte immediately following the image of the last floating-point data register loaded. The order of the transfer is the same as for the control addressing modes: FP7–FP0.

Status Register: Not Affected. Note that the FMOVEM instruction provides the only mechanism for moving a floating-point data item between the FPCP and memory without performing any data conversions or affecting the condition code and exception status bits.

4

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
1	1	dr	MODE		0	0	0	REGISTER LIST							

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for the operation:

Memory-to-Register

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
– (An)	—	—
{d16,An}	101	reg. number:An
{d8,An,Xn}	110	reg. number:An
{bd,An,Xn}	110	reg. number:An
{[bd,An,Xn],od}	110	reg. number:An
{[bd,An],Xn,od}	110	reg. number:An

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	—	—
{d16,PC}	111	010
{d8,PC,Xn}	111	011
{bd,PC,Xn}	111	011
{[bd,PC,Xn],od}	111	011
{[bd,PC],Xn,od}	111	011

Register-to-Memory

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	—	—
(An) +	—	—			
-(An)	100	reg. number:An	(d16,PC)	—	—
(d16,An)	101	reg. number:An	(dg,PC,Xn)	—	—
(dg,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	([bd,PC,Xn],od)	—	—
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	—	—
([bd,An],Xn,od)	110	reg. number:An			

dr Field — Specifies the direction of the transfer.

0 — Move the listed registers from memory to the FPCP.

1 — Move the listed registers from the FPCP to memory.

Mode Field — Specifies the type of the register list and addressing mode.

00 — Static register list, predecrement addressing mode.

01 — Dynamic register list, predecrement addressing mode.

10 — Static register list, postincrement or control addressing mode.

11 — Dynamic register list, postincrement or control addressing mode.

Register List Field:

Static list — contains the register select mask; if a register is to be moved, the corresponding bit in the mask is set as shown below, otherwise it is clear.

Dynamic list — contains the main processor data register number, rrr, as shown below:

Register List Format

Static, -(An) — FP7 FP6 FP5 FP4 FP3 FP2 FP1 FP0

Static, (An) + or

Control — FP0 FP1 FP2 FP3 FP4 FP5 FP6 FP7

Dynamic — 0 r r r 0 0 0 0

The format of the dynamic list mask is the same as for the static list and is contained in the least significant eight bits of the specified main processor data register.

Programming Note: This instruction provides a very useful feature, dynamic register list specification, that can significantly enhance system performance. If the calling conventions used for procedure calls utilize the dynamic register list feature, the number of floating-point data registers saved and restored can be reduced. A minimum of six bus cycles is required to load or save a floating-point data register (more if the memory address is not long word aligned). Thus, a minimum of 36 clock cycles (2×6 bus cycles×3 clocks per bus cycle) is eliminated from the procedure call and return overhead for each register not saved and restored unnecessarily.

In order to utilize the dynamic register specification feature of the FMOVEM instruction, both the calling and the called procedures must be written to communicate information about register usage. When one procedure calls another, a register mask must be passed to the called procedure to indicate which registers must not be altered upon return to the calling procedure. The called procedure then saves only those registers

that are modified and are already in use. There are several techniques that can be used to utilize this mechanism, and an example follows.

In this example, a convention is defined by which each called procedure is passed a word mask in D7 that identifies all floating-point registers in use by the calling procedure. Bits 15–8 identify the registers in the order FP0–FP7, and bits 0–7 identify the registers in the order FP0–FP7 (the two masks are required due to the different transfer order used by the predecrement and postincrement addressing modes). The code used by the calling procedure consists of simply moving the mask (which is generated at compile time) for the floating-point data registers currently in use into D7:

Calling procedure . . .

```
MOVE.W #ACTIVE,D7    Load the list of FP registers that are in use
BSR     PROC_2
```

The entry code for all other procedures computes two masks. The first mask identifies the registers in use by the calling procedure that are used by the called procedure (and therefore saved and restored by the called procedure). The second mask identifies the registers in use by the calling procedure that are used by the called procedure (and therefore not saved on entry). The appropriate registers are then stored along with the two masks:

Called procedure . . .

```
MOVE.W D7,D6          Copy the list of active registers
AND.W  #WILL_USE,D7   Generate the list of doubly-used registers
FMOVEM D7,-(A7)        Save those registers
MOVE.W D7,-(A7)        Save the register list
EOR.W  D7,D6          Generate the list of not saved active registers
MOVE.W D6,P(A7)        Save it for later use
```

If the second procedure calls a third procedure, a register mask is passed to the third procedure that indicates which registers must not be altered by the third procedure. This mask identifies any registers in the list from the first procedure that were not saved by the second procedure, plus any registers used by the second procedure that must not be altered by the third procedure. An example of the calculation of this mask is:

Nested calling sequence . . .

```
MOVE.W UNSAVED        Load the list of active registers not saved at entry
      (A7),D7
OR.W   #ACTIVE,D7      Combine with those active at this time
BSR    PROC_3
```

Upon return from a procedure, the restoration of the necessary registers follows the same convention, and the register mask generated during the save operation on entry is used to restore the required floating-point data registers:

Return to caller . . .

```
ADDQ.L #2,A7          Discard the list of registers not saved
MOVE.B (A7)+,D7        Get the saved register list (pop word, use byte)
FMOVEM (A7)+,D7        Restore the registers
.
.
.
RTS                    Return to the
                       calling routine
```

Operation: Register List $\leftrightarrow$ Destination
Source $\leftrightarrow$ Register List

Assembler Syntax: FMOVEM.L <list>,<ea>
FMOVEM.L <ea>,<list>

<list> A list of any combination of the three floating-point system control registers (FPCR, FPSR and FPIAR) with individual register names separated by a slash (/).

4

Attributes: Size = (Long)

Description: Moves one or more 32-bit values into or out of the specified system control registers. Any combination of the three system control registers may be specified. The registers are always moved in the same order, regardless of the addressing mode used; with the FPCR moved first, followed by the FPSR, and the FPIAR moved last (if a register is not selected for the transfer, the relative order of the transfer of the other registers is the same). The first register is transferred between the FPCP and the specified address, with successive registers located up through higher addresses.

When more than one register is moved, the memory or memory-alterable addressing modes are allowed as shown in the addressing mode tables. If the addressing mode is predecrement, the address register is first decremented by the total size of the register images to be moved (i.e., four times the number of registers) and then the registers are transferred starting at the resultant address. For the postincrement addressing mode, the selected registers are transferred to or from the specified address, and then the address register is incremented by the total size of the register images transferred. If a single system control register is selected, the data register direct addressing mode may be used; or, if the only register selected is the FPIAR, then the address register direct addressing mode is allowed. Note that if a single register is selected, the opcode generated is the same as for the FMOVE single system control register instruction.

Status Register: Is changed only if the destination list includes the FPSR; in which case all bits are modified to reflect the value of the source register image.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
1	0	dr	REGISTER LIST			0	0	0	0	0	0	0	0	C	C

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for the operation:

Memory-to-Register

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An**	001	reg. number:An
{An}	010	reg. number:An
{An} +	011	reg. number:An
– {An}	100	reg. number:An
{d ₁₆ ,An}	101	reg. number:An
{dg,An,Xn}	110	reg. number:An
{bd,An,Xn}	110	reg. number:An
{[bd,An,Xn],od}	110	reg. number:An
{[bd,An],Xn,od}	110	reg. number:An

*Only if a single FPCR is selected.

**Only if the FPIAR is the single register selected.

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	111	100
{d ₁₆ ,PC}	111	010
{dg,PC,Xn}	111	011
{bd,PC,Xn}	111	011
{[bd,PC,Xn],od}	111	011
{[bd,PC],Xn,od}	111	011

4

Register-to-Memory

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An**	001	reg. number:An
{An}	010	reg. number:An
{An} +	011	reg. number:An
– {An}	100	reg. number:An
{d ₁₆ ,An}	101	reg. number:An
{dg,An,Xn}	110	reg. number:An
{bd,An,Xn}	110	reg. number:An
{[bd,An,Xn],od}	110	reg. number:An
{[bd,An],Xn,od}	110	reg. number:An

*Only if a single FPCR is selected.

**Only if the FPIAR is the single register selected.

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	—	—
{d ₁₆ ,PC}	—	—
{dg,PC,Xn}	—	—
{bd,PC,Xn}	—	—
{[bd,PC,Xn],od}	—	—
{[bd,PC],Xn,od}	—	—

dr Field — Specifies the direction of the transfer.

0 — Move the listed registers from memory to the FPCP.

1 — Move the listed registers from the FPCP to memory.

Register List Field: — Contains the register select mask; if a register is to be moved, the corresponding bit in the list is set, otherwise it is clear.

Bit Number	—	12	11	10
Register	—	FPCR	FPSR	FPIAR

Operation: Source $\times$ FPn $\rightarrow$ FPn

Assembler FMUL.<fmt> <ea>,FPn

Syntax: FMUL.X FPM,FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and multiplies that number by the number in the destination floating-point data register. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Multiply		+inf	-inf
					-inf	+inf
Zero	+	-	+0.0	-0.0	NaN ¹	
			-0.0	+0.0		
Infinity	+	-	+inf	-inf	NaN ¹	
			-inf	+inf		

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set for 0 $\times$ infinity; cleared otherwise
OVFL	Refer to 6.1.4 Overflow.
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	1	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: $-(\text{Source}) \uparrow \text{FPn}$

Assembler FNEG.<fmt> <ea>,FPn

Syntax: FNEG.X FPm,FPn
FNEG.X FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and inverts the sign of the mantissa. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Negate		-0.0	-0.0	-inf	-inf

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Cleared
OVFL	Cleared
UNFL	If source is an extended precision denormalized number, refer to 6.1.5 Underflow; cleared otherwise.
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
{An}	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation:	None
Assembler Syntax:	FNOP
Attributes:	Unsize

4

Description: This instruction does not perform any explicit operation. However, it is useful to force synchronization of the FPCP with a main processor, or to force processing of pending exceptions. The synchronization function is inherent in the way that the FPCP uses the M68000 Family coprocessor interface. For most FPCP instructions, the main processor is allowed to continue with the execution of the next instruction once the FPCP has any operands needed for an operation, thus supporting concurrent execution of floating-point and integer instructions. However, if the main processor attempts to initiate the execution of a new floating-point instruction in the MC68881 before the previous one is completed, the main processor is forced to wait until that instruction execution is finished before proceeding with the new instruction. FNOP is treated in the same way as other instructions and thus cannot be executed until the previous floating-point instruction is completed, and the main processor is synchronized with the MC68881.

The MC68882 may not wait to begin execution of another floating-point instruction until it has completed execution of the current instruction. However, the FNOP instruction synchronizes the coprocessor and MPU by causing the MPU to wait until the current instruction (or both instructions) have completed.

The FNOP instruction also forces the processing of exceptions pending from the execution of previous instructions. This is also inherent in the way that the FPCP utilizes the M68000 Family coprocessor interface. Once the FPCP has received the input operand for an arithmetic instruction, it always releases the main processor to execute the next instruction (regardless of whether or not concurrent execution is prevented for the instruction due to tracing) without reporting the exception during the execution of that instruction. Then, when the main processor attempts to initiate the execution of the next FPCP instruction, a pre-instruction exception may be reported to initiate exception processing for an exception that occurred during a previous instruction. By using the FNOP instruction, the user can force any pending exceptions to be processed without performing any other operations.

Status Register: Not Affected

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. **Freescal** assemblers default to ID=1 for the FPCP.

NOTE

FNOP uses the same opcode as the FBcc.W <label> instruction, with cc=F (non-trapping false) and <label>=*+2 (which results in a displacement of 0).

Operation: IEEE Remainder of (FPn (÷) Source) ÷ FPn

Assembler: FREM.<fmt> <ea>,FPn

Syntax: FREM.X FPm,FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the modulo remainder of the number in the destination floating-point data register, using the source operand as the modulus. Stores the result in the destination floating-point data register, and stores the sign and seven least significant bits of the quotient in the FPSR quotient byte (the quotient is the result of FPn (÷) Source). The IEEE remainder function is defined as:

$$FPn - (Source \times N)$$

where:

$N = \text{INT}(FPn \div \text{Source})$ in the round-to-nearest mode

The FREM function is not defined for a source operand equal to zero or for a destination operand equal to infinity. Note that this function is not the same as the FMOD instruction, which uses the round-to-zero mode and thus returns a remainder that is different from the remainder required by the *IEEE Specification for Binary Floating-Point Arithmetic*.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	IEEE Remainder		NaN ¹	
Zero	+	-	+0.0		NaN ¹	
	-	-	-0.0		+0.0	
Infinity	+	-	NaN ¹		NaN ¹	
	-	-	NaN ¹		NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. Returns the value of FPn before the operation. However, the result is processed by the normal instruction termination procedure to round it as required. Thus, an underflow and/or inexact result may occur if the rounding precision has been changed to a smaller size since the FPn value was loaded.
3. If either operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Loaded with the sign and least significant seven bits of the quotient (FPn (÷) Source). The sign of the quotient is the exclusive OR of the sign bits of the source and destination operands.

Exception Byte: BSUN Cleared
 SNAN Refer to 4.5.4 NaNs.
 OPERR Set if the source is zero, or the destination is infinity; cleared otherwise

OVFL Cleared
 UNFL Refer to **6.1.5 Underflow**.
 DZ Cleared
 INEX2 Cleared
 INEX1 If <fmt> is Packed, refer to **6.1.8 Inexact Result on Decimal Input**; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	0	1

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
{d16,An}	101	reg. number:An	{d16,PC}	111	010
{d8,An,Xn}	110	reg. number:An	{d8,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000 L Long Word Integer
 001 S Single Precision Real
 010 X Extended Precision Real
 011 P Packed Decimal Real
 100 W Word Integer
 101 D Double Precision Real
 110 B Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: If in supervisor state
then FPCP State Frame → Internal State
else TRAP

Assembler

Syntax: FRESTORE <ea>

Attributes: Unsized, privileged.

Description: Aborts the execution of any operation in progress, and loads a new internal state from the state frame located at the effective address. The first word at the specified address is the format word of the state frame, which specifies the size of the frame and the revision number of the FPCP that created it. The MPU writes the first word to the FPCP Restore CIR to initiate the restore operation, and then reads the response CIR to verify that the FPCP recognizes the format word as valid. If the format word is invalid for the FPCP (either because the size of the frame is not recognized, or the revision number does not match the revision of the processor), the MPU is instructed to take a format exception. The MPU then writes an abort to the control CIR, and the FPCP enters the IDLE state. If the format word is valid, the appropriate state frame is loaded, starting at the specified location and proceeding through higher addresses.

The FRESTORE instruction does not normally affect the programmer's model registers of the FPCP (except for the NULL state size, as described below), but is used only to restore the user invisible portion of the machine. The FRESTORE instruction is used with the FMOVE instruction to perform a full context restoration of the FPCP, including the floating-point data registers and system control registers. In order to accomplish a complete restoration, the FMOVE instructions are first executed to load the programmer's model, followed by the FRESTORE instruction to load the internal state and continue any previously suspended operation. Refer to 6.4 CONTEXT SWITCHING for more information.

The current implementation of the FPCP supports three state frames. Refer to 6.4.2 **State Frames** for the exact format of these state frames.

NULL: This state frame is four bytes long, with a format word of \$0000. A FRESTORE operation with this size state frame is equivalent to a hardware reset of the FPCP. The programmer's model is set to the reset state, with nonsignaling NaNs in the floating-point data registers and zeros in the FPCR, FPSR and FPIAR. (Thus, it is unnecessary to load the programmer's model before this operation.)

IDLE: This state frame is 28 (\$1C) bytes long in the MC68881, and 60 (\$3C) bytes long in the MC68882. An FRESTORE operation with this size state frame causes the FPCP to be restored to the idle state, waiting for the initiation of the next instruction. Exceptions that were pending before the execution of the previous FSAVE instruction are pending following the execution of the FRESTORE instruction. The programmer's model is not affected by loading this type of state frame.

BUSY: This state frame is 184 (\$B8) bytes long in the MC68881 and 216 (\$D8) bytes long in the MC68882. An FRESTORE operation with this size state frame causes the FPCP to be restored to the busy state, executing the instruction that was suspended by a previous FSAVE operation. The programmer's model is not affected by loading this type of state frame (although the completion of the suspended instruction after the restore is executed may modify the programmer's model).

Status Register: Cleared if the state size is NULL, otherwise not affected

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	1	EFFECTIVE ADDRESS MODE			REGISTER		

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for the state frame. Only postincrement or control addressing modes are allowed as shown:

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	—	—
(d ₁₆ ,An)	101	reg. number:An
(dg,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
[(bd,An,Xn),od]	110	reg. number:An
[(bd,An),Xn,od]	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	—	—
(d ₁₆ ,PC)	111	010
(dg,PC,Xn)	111	011
(bd,PC,Xn)	111	011
[(bd,PC,Xn),od]	111	011
[(bd,PC),Xn,od]	111	011

Operation: If in supervisor state
then FPCP Internal State → State Frame
else TRAP

Assembler

Syntax: FSAVE <ea>

Attributes: Unsized, privileged.

Description: Suspends the execution of any operation in progress, and saves the internal state in a state frame located at the effective address. After the save operation, the FPCP is in the idle state, waiting for the execution of the next instruction. The first word written to the state frame is the format word, which specifies the size of the frame and the revision number of the FPCP. The MPU initiates the FSAVE instruction by reading the FPCP save CIR, which is encoded with a format word that indicates the appropriate action to be taken by the main processor. The current implementation of the FPCP always returns one of five responses in the save CIR:

Value	Definition
\$0018	Save NULL state frame
\$0118	Not ready, come again
\$0218	Illegal, take format exception
\$XX18	Save IDLE state frame
\$XXB4	Save BUSY state frame

where:

XX is the FPCP version number.

The Not Ready format word indicates that the FPCP is not prepared to perform a state save and that the MPU should process interrupts, if necessary, and then re-read the save CIR. The FPCP uses this format word to cause the main processor to wait while an internal operation is completed, if possible, in order to allow an IDLE frame to be saved rather than a BUSY frame. The Illegal format word is used to abort an FSAVE instruction that is attempted while the FPCP is executing a previous FSAVE instruction. All other format words cause the MPU to save the indicated state frame at the specified address. For state frame details see **6.4.2 State Frames**. These state frames are defined as follows:

NULL: This state frame is four bytes long. An FSAVE instruction that generates this size state frame indicates that the FPCP state has not been modified since the last hardware reset or FRESTORE instruction with a NULL state frame. This indicates that the programmer's model is in the reset state, with nonsignaling NaNs in the floating-point data registers and zeros in the FPCR, FPSR, and FPIAR. (Thus, it is not necessary to save the programmer's model.)

- IDLE:** This state frame is 28 (\$1C) bytes long in the MC68881, and 60 (\$3C) bytes long in the MC68882. An FSAVE instruction that generates this size state frame indicates that the FPCP was in an idle condition, waiting for the initiation of the next instruction. Any exceptions that were pending are saved in the frame and are then cleared internally. Thus, the pending exceptions are not reported until after a subsequent FRESTORE instruction loads the state frame. In addition to being used for context switching, this frame may be used by exception handler routines, since it contains the value of the operand that caused the last floating-point exception.
- BUSY:** This state frame is 184 (\$B8) bytes long in the MC68881, and 216 (\$D8) bytes long in the MC68882. An FSAVE instruction that generates this size state frame indicates that the FPCP was at a point within an instruction where it was necessary to save the entire internal state of the processor. This frame size is only used when absolutely necessary because of the large size of the frame and the amount of time required to transfer it. The action of the FPCP when this state frame is saved is the same as for the IDLE state frame.

The FSAVE does not save the programmer's model registers of the FPCP, but is used to save only the user invisible portion of the machine. The FSAVE instruction may be used with the FMOVEM instruction to perform a full context save of the FPCP including the floating-point data registers and system control registers. In order to accomplish a complete context save, an FSAVE instruction is first executed to suspend the current operation and save the internal state, followed by the appropriate FMOVEM instructions to store the programmer's model. Refer to **6.4 CONTEXT SWITCHING** for more information.

Status Register: Not affected

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	0	EFFECTIVE ADDRESS MODE REGISTER					

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for the state frame. Only predecrement or control-alterable addressing modes are allowed as shown:

4

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
{An}	010	reg. number:An
{An}+	—	—
— {An}	100	reg. number:An
{d16,An}	101	reg. number:An
{d8,An,Xn}	110	reg. number:An
{bd,An,Xn}	110	reg. number:An
{[bd,An,Xn],od}	110	reg. number:An
{[bd,An],Xn,od}	110	reg. number:An

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	—	—
{d16,PC}	—	—
{d8,PC,Xn}	—	—
{bd,PC,Xn}	—	—
{[bd,PC,Xn],od}	—	—
{[bd,PC],Xn,od}	—	—

Operation: $FP_n \times INT(2^{Source}) \rightarrow FP_n$

Assembler Syntax: `FSCALE.<fmt> <ea>,FPn`
`FSCALE.X FPm,FPn`

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to an integer (if necessary) and adds that integer to the destination exponent. Stores the result in the destination floating-point data register. This function has the effect of multiplying the destination by 2^{Source} , but is much faster than a multiply operation when the source is an integer value.

4

The FPCP assumes that the scale factor is an integer value before the operation is executed. If not, the value is chopped (i.e., rounded using the round-to-zero mode) to an integer before it is added to the exponent. When the absolute value of the source operand is $(\geq) 2^{14}$, an overflow or underflow always results.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Scale Exponent		FP_n^1	
Zero	+	-	+0.0 -0.0		NaN ²	
	+	-	+inf -inf		NaN ²	
Infinity	+	-	+inf -inf		NaN ²	
	+	-	+inf -inf		NaN ²	

NOTES:

1. Returns the value FP_n before the operation. However, the result is processed by the normal instruction termination procedure to round it as required. Thus, an underflow and/or inexact result may occur if the rounding precision has been changed to a smaller size since the FP_n value was loaded.
2. Sets the OPERR bit in the FPSR exception byte.
3. If the source operand is a NaN, refer to **4.5.4 NaNs** for more information.

Status Register:

Condition Codes: Affected as described in **4.5.5.1 SETTING FLOATING-POINT CONDITION CODES**

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs .
OPERR	Set if the source operand is (+ or -)infinity; cleared otherwise
OVFL	Refer to 6.1.4 Overflow .
UNFL	Refer to 6.1.5 Underflow .

DZ Cleared
 INEX2 Cleared
 INEX1 If <fmt> is Packed, refer to **6.1.8 Inexact Result on Decimal Input**; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

4

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M = 0, this field is unused, and should be all zeros.

If R/M = 1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An) +	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(d8,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d16,PC)	111	010
(d8,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

Operation: If (condition true)
then 1s → Destination
else 0s → Destination

Assembler

Syntax: FScc.<size> <ea>

Attributes: Size = (Byte)

Description: If the specified floating-point condition is true, sets the byte integer operand at the destination to TRUE (all ones), otherwise sets the byte to FALSE (all zeros). The conditional specifier cc may select any one of the 32 floating-point conditional tests as described in **4.4 CONDITIONAL TEST DEFINITIONS**.

Status Register:

Condition Codes: Not affected

Quotient Byte: Not affected

Exception Byte: BSUN Set if the NAN condition code is set and the condition selected is an IEEE nonaware test

SNAN	Not Affected
OPERR	Not Affected
OVFL	Not Affected
UNFL	Not Affected
DZ	Not Affected
INEX2	Not Affected
INEX1	Not Affected

Accrued Exception Byte: The IOP bit is set if the BSUN bit is set in the exception byte. No other bit is affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	EFFECTIVE ADDRESS MODEREGISTER					
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Specifies the addressing mode for the byte integer operand:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	—	—
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	—	—
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	—	—
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	—	—
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	—	—
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	—	—

Conditional Predicate Field — Specifies one of 32 conditional tests as defined in 4.4
CONDITIONAL TEST DEFINITIONS.

NOTE

When a BSUN exception occurs, a pre-instruction exception is taken by the main processor. If the exception handler returns without modifying the image of the PC on the stack frame (to point to the instruction following the FScc), then it must clear the cause of the exception (by clearing the NAN bit or disabling the BSUN trap) or the exception occurs again immediately upon return to the routine that caused the exception.

Operation: FPN ($\div$) Source $\rightarrow$ FPN

Assembler FSGLDIV.<fmt> <ea>,FPn

Syntax: FSGLDIV.X FPM,FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and divides that number into the number in the destination floating-point data register. Stores the result in the destination floating-point data register, rounded to single precision (regardless of the current rounding precision). This function is undefined for $0(\div)0$ and infinity($\div$)infinity.

Both the source and destination operands are assumed to be representable in the single precision format. If either operand requires more than 24 bits of mantissa to be accurately represented, the accuracy of the result is not guaranteed. Furthermore, the result exponent may exceed the range of single precision, regardless of the rounding precision selected in the FPCR mode control byte. Refer to **4.5.5.2 UNDERFLOW, ROUND, OVERFLOW** for more information.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Divide (Single Precision)	$\pm \text{inf}^1$ $-\text{inf}^1$	± 0.0 -0.0	± 0.0 -0.0
Zero	+	-	± 0.0 -0.0	-0.0 ± 0.0	NaN ²	
Infinity	+	-	$\pm \text{inf}$ $-\text{inf}$	$-\text{inf}$ $\pm \text{inf}$	NaN ²	

NOTES:

1. Sets the DZ bit in the FPSR exception byte.
2. Sets the OPERR bit in the FPSR exception byte.
3. If the source operand is a NaN, refer to **4.5.4 NaNs** for more information.

Status Register:

Condition Codes:	Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES	
Quotient Byte:	Not affected	
Exception Byte:	BSUN	Cleared
	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Set for $0(\div)0$ or infinity($\div$)infinity
	OVFL	Refer to 6.1.4 Overflow .
	UNFL	Refer to 6.1.5 Underflow .
	DZ	Set if the source is zero and the destination is in range; cleared otherwise
	INEX2	Refer to 6.1.7 Inexact Result .
	INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input ; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	0	0

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An) +	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPN.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: Source $\times$ FPn $\rightarrow$ FPn

Assembler: FSGLMUL.<fmt>

Syntax: FSGLMUL.X
<ea>,FPn
FPm,FPn

4

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and multiplies that number by the number in the destination floating-point data register. Stores the result in the destination floating-point data register, rounded to single precision (regardless of the current rounding precision).

Both the source and destination operands are assumed to be representable in the single precision format. If either operand requires more than 24 bits of mantissa to be accurately represented, the accuracy of the result is not guaranteed. Furthermore, the result exponent may exceed the range of single precision, regardless of the rounding precision selected in the FPCR mode control byte. Refer to **4.5.5.2 UNDERFLOW, ROUND, OVERFLOW** for more information.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
In Range	+	-	Multiply (Single Precision)	+0.0 -0.0	+inf -inf	-inf +inf
Zero	+	-	+0.0 -0.0	-0.0 +0.0	NaN ¹	
Infinity	+	-	+inf -inf	-inf +inf	NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to **4.5.4 NaNs** for more information.

Status Register:

Condition Codes:	Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES	
Quotient Byte:	Not affected	
Exception Byte:	BSUN	Cleared
	SNAN	Refer to 4.5.4 NaNs .
	OPERR	Set if one operand is zero and the other is infinity; cleared otherwise
	OVFL	Refer to 6.1.4 Overflow .
	UNFL	Refer to 6.1.5 Underflow .
	DZ	Cleared
	INEX2	Refer to 6.1.7 Inexact Result .

INEX1

If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	1	1

4

Instruction Fields:

Coprorocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(d8,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
((bd,An,Xn),od)	110	reg. number:An
((bd,An,Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d16,PC)	111	010
(d8,PC,Xn)	111	011
(bd,PC,Xn)	111	011
((bd,PC,Xn),od)	111	011
((bd,PC,Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

Operation: Sine of Source $\rightarrow$ FPN

Assembler FSIN.<fmt> <ea>,FPn

Syntax: FSIN.X FPM,FPn

FSIN.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates the sine of that number. Stores the result in the destination floating-point data register. This function is not defined for source operands of $(\pm)\infty$. If the source operand is not in the range of $[-2\pi \dots +2\pi]$, the argument is reduced to within that range before the sine is calculated. However, large arguments may lose accuracy during reduction, and very large arguments (greater than approximately 10^{20}) lose all accuracy. The result is in the range of $[-1 \dots +1]$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Sine		+0.0	-0.0	NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source is $(+ \text{ or } -)\infty$; cleared otherwise.
OVFL	Cleared
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(dg,An,Xn)	110	reg. number:An	(dg,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPN.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, then the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Sine of Source $\rightarrow$ FPs
Cosine of Source $\rightarrow$ FPc

Assembler Syntax: FSINCOS.<fmt> <ea>,FPc:FPs
FSINCOS.X FPr,FPc:FPs

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

4

Description: Converts the source operand to extended precision (if necessary) and calculates both the sine and the cosine of that number. Calculates both functions simultaneously; thus, this instruction is significantly faster than performing separate FSIN and FCOS instructions. Loads the sine result into the destination floating-point data register, FPs, and the cosine result into the destination floating-point data register FPc. Sets the condition code bits according to the sine result. If FPs and FPc are specified to be the same register, the cosine result is first loaded into the register and then is overwritten with the sine result. This function is not defined for source operands of $(\pm)$ infinity.

If the source operand is not in the range of $(-2\pi \dots +2\pi)$, the argument is reduced to within that range before the sine and cosine are calculated. However, large arguments may lose accuracy during reduction, and very large arguments (greater than approximately 10^{20}) lose all accuracy. The results are in the range of $(-1 \dots +1)$.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
FPs	Sine		+0.0	-0.0	NaN ¹	
FPc	Cosine		+1.0		NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES (for the sine result)

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source is $(\pm)$ infinity; cleared otherwise
OVFL	Cleared
UNFL	Set if a sine underflow occurs, in which case the cosine result is 1. Cosine cannot underflow. Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.

INEX1

If <fmt> is Packed, refer to **6.1.8 Inexact Result on Decimal Input**; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER, FPs			0	1	1	0	DESTINATION REGISTER, FPC		

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d16,An)	101	reg. number:An	(d16,PC)	111	010
(d8,An,Xn)	110	reg. number:An	(d8,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011
([bd,An,Xn],od)	110	reg. number:An	([bd,PC,Xn],od)	111	011
([bd,An],Xn,od)	110	reg. number:An	([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPC.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register, FPc Field — Specifies the destination floating-point data register, FPc. The cosine result is stored in this register.

Destination Register, FPs Field — Specifies the destination floating-point data register, FPs. The sine result is stored in this register. If FPc and FPs specify the same floating-point data register, the sine result is stored in the register, and the cosine result is discarded.

4

If $R/M=0$ and the source register field is equal to either of the destination register fields, the input operand is taken from the specified floating-point data register, and the appropriate result is written into the same register.

Operation: Hyperbolic Sine of Source $\rightarrow$ FPn

Assembler Syntax: FSINH.<fmt> <ea>,FPn
FSINH.X FPm,FPn
FSINH.X FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the hyperbolic sine of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Hyperbolic Sine		+0.0	-0.0	+inf	-inf

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
(An)	010	reg. number:An	#<data>	111	100
(An)+	011	reg. number:An			
-(An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
{dg,An,Xn}	110	reg. number:An	{dg,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Square Root of Source ♦ FPn

Assembler FSQRT.<fmt> <ea>,FPn

Syntax: FSQRT.X FPm,FPn

FSQRT.X FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the square root of that number. Stores the result in the destination floating-point data register. This function is not defined for negative operands.

4

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	$\sqrt{x}$	NAN ¹	+0.0	-0.0	+inf	NAN ¹

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if the source operand is not zero and is negative; cleared otherwise
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS					
				SOURCE SPECIFIER			DESTINATION REGISTER			MODE		REGISTER			
0	R/M	0							0	0	0	0	1	0	0

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number:An	#<data>	111	100
{An}+	011	reg. number:An			
-(An)	100	reg. number:An			
{d16,An}	101	reg. number:An	{d16,PC}	111	010
{dg,An,Xn}	110	reg. number:An	{dg,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number:An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number:An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number:An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: FPN – Source ∇ FPN

Assembler FSUB.<fmt> <ea>,FPn

Syntax: FSUB.X FPM,FPn

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and subtracts that number from the number in the destination floating-point data register. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source		In Range		Zero		Infinity	
		+	-	+	-	+	-
In Range	+	Subtract		Subtract		- inf	+ inf
	-	Subtract		Subtract		- inf	+ inf
Zero	+	Subtract		+0.0 ¹	-0.0	- inf	+ inf
	-	Subtract		-0.0	+0.0 ¹	- inf	+ inf
Infinity	+	+ inf		+ inf		NAN ²	- inf
	-	- inf		- inf		- inf	NAN ²

NOTES:

1. Returns +0.0 in rounding modes RN, RZ, and RP; returns -0.0 in RM.
2. Sets the OPERR bit in the FPSR exception byte.
3. If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs.
OPERR	Set if both the source and destination are like-signed infinities; cleared otherwise
OVFL	Refer to 6.1.4 Overflow.
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	1	0	0	0

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

Operation: Tangent of Source $\rightarrow$ FPn

Assembler FTAN.<fmt> <ea>,FPn

Syntax: FTAN.X FpM,FPn
FTAN.X FpN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the tangent of that number. Stores the result in the destination floating-point data register. This function is not defined for source operands of $(\pm)$ infinity. If the source operand is not in the range of $[-\pi/2 \dots +\pi/2]$, the argument is reduced to within that range before the tangent is calculated. However, large arguments may lose accuracy during reduction, and very large arguments (greater than approximately 10^{20}) lose all accuracy.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Tangent		+ 0.0	- 0.0	NaN ¹	

NOTES:

1. Sets the OPERR bit in the FPSR exception byte.
2. If the source operand is a NaN, refer to 4.5.4 NaNs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NaNs.
OPERR	Set if the source is $(\pm)$ infinity; cleared otherwise
OVFL	Refer to 6.1.4 Overflow.
UNFL	Refer to 6.1.5 Underflow.
DZ	Cleared
INEX2	Refer to 6.1.7 Inexact Result.
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	1	1

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M = 0, this field is unused, and should be all zeros.

If R/M = 1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn*	000	reg. number: Dn	{xxx}.W	111	000
An	—	—	{xxx}.L	111	001
{An}	010	reg. number: An	#<data>	111	100
{An}+	011	reg. number: An			
-(An)	100	reg. number: An			
{d16,An}	101	reg. number: An	{d16,PC}	111	010
{dg,An,Xn}	110	reg. number: An	{dg,PC,Xn}	111	011
{bd,An,Xn}	110	reg. number: An	{bd,PC,Xn}	111	011
{[bd,An,Xn],od}	110	reg. number: An	{[bd,PC,Xn],od}	111	011
{[bd,An],Xn,od}	110	reg. number: An	{[bd,PC],Xn,od}	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M = 0, specifies the source floating-point data register, FPM.

If R/M = 1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M = 0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: Hyperbolic Tangent of Source ∇ FPn

Assembler FTANH.<fmt> <ea>,FPn

Syntax: FTANH.X FPm,FPn
FTANH.X FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates the hyperbolic tangent of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	Hyperbolic Tangent		+0.0	-0.0	+1.0	-1.0

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Cleared
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	0	0	1

4

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(dg,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
((bd,An,Xn),od)	110	reg. number:An
((bd,An),Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d16,PC)	111	010
(dg,PC,Xn)	111	011
(bd,PC,Xn)	111	011
((bd,PC,Xn),od)	111	011
((bd,PC),Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPM.

If R/M=0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: 10Source $\rightarrow$ FPn

Assembler: FTENTOX.<fmt> <ea>,FPn

Syntax: FTENTOX.X FPM,FPn
FTENTOX.X FPN

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates 10 to the power of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	10 ^x		+1.0		+inf	+0.0

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(d8,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	111	100
(d16,PC)	111	010
(d8,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

If R/M=0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

Operation: If condition true, then TRAP

Assembler FTRAPcc

Syntax: FTRAPcc.W #<data>
FTRAPcc.L #<data>

Attributes: Size = (Word, Long)

Description: If the selected condition is true, the main processor initiates exception processing. A vector number is generated to reference the TRAPcc exception vector. The stacked program counter points to the next instruction. If the selected condition is not true, no operation is performed, and execution continues with the next instruction in sequence. The immediate data operand is placed in the word(s) following the conditional predicate word and is available for user definition for use within the trap handler.

The conditional specifier cc selects one of the 32 conditional tests defined in 4.4 **CONDITIONAL TEST DEFINITIONS**.

Status Register:

Condition Codes: Not affected

Quotient Byte: Not affected

Exception Byte: BSUN Set if the NAN condition code is set and the condition selected is an IEEE nonaware test
SNAN Not Affected
OPERR Not Affected
OVFL Not Affected
UNFL Not Affected
DZ Not Affected
INEX2 Not Affected
INEX1 Not Affected

Accrued Exception Byte: The IOP bit is set if the BSUN bit is set in the exception byte. No other bit is affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID				0	0	1	1	1	1	MODE	
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					
16-BIT OPERAND OR MOST SIGNIFICANT WORD OF 32-BIT OPERAND (IF NEEDED)															
LEAST SIGNIFICANT WORD OR 32-BIT OPERAND (IF NEEDED)															

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID = 1 for the FPCP.

Mode Field — Specifies the form of the instruction.

010 — The instruction is followed by a word operand.

011 — The instruction is followed by a long word operand.

100 — The instruction has no operand.

Conditional Predicate Field — Specifies one of 32 conditional tests as described in 4.4 **CONDITIONAL TEST DEFINITIONS**.

Operand Field — Contains an optional word or long word operand that is user defined.

NOTE

When a BSUN exception occurs, a pre-instruction exception is taken by the main processor. If the exception handler returns without modifying the image of the PC on the stack frame (to point to the instruction following the FTRAPcc), it must clear the cause of the exception (by clearing the NAN bit or disabling the BSUN trap) or the exception occurs again immediately upon return to the routine that caused the exception.

Operation: Condition Codes for Operand ♦ FPCC

Assembler Syntax: FTST.<fmt> <ea>
FTST.X FPM

Attributes: Format = (Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and sets the condition code bits according to the data type of the result.

Operation Table: The contents of this table differ from the other operation tables. A letter in an entry of this table indicates that the designated condition code bit is always set by the FTST operation. All unspecified condition code bits are cleared during the operation.

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	none	N	Z	NZ	I	NI

NOTES:

1. If the source operand is a NAN, set the NAN condition code bit.
2. If the source operand is a SNAN, set the SNAN bit in the FPSR exception byte.

Status Register:

Condition Codes: Affected as described in **4.5.5.1 SETTING FLOATING-POINT CONDITION CODES**

Quotient Byte: Not affected

Exception Byte:

BSUN	Cleared
SNAN	Refer to 4.5.4 NANs .
OPERR	Cleared
OVFL	Cleared
UNFL	Cleared
DZ	Cleared
INEX2	Cleared
INEX1	If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input ; cleared otherwise.

Accrued Exception Byte: Affected as described in **6.1.10 IEEE Exception and Trap Compatibility**

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	1	1	0	1	0

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

4

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(b _d ,An,Xn)	110	reg. number:An
[(b _d ,An,Xn),od]	110	reg. number:An
[(b _d ,An),Xn,od]	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
#<data>	111	100
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011
(b _d ,PC,Xn)	111	011
[(b _d ,PC,Xn),od]	111	011
[(b _d ,PC),Xn,od]	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPM.

If R/M=1, specifies the source data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Since the FPCP uses a common command word format for all of the arithmetic instructions (including FTST), this field is treated in the same manner for FTST as for the other arithmetic instructions, even though the destination register is not modified. This field should be set to zero in order to maintain compatibility with future devices, although the FPCP does not signal an illegal instruction trap if it is not zero.

Operation: 2Source $\rightarrow$ FPn

Assembler: FTWOTOX.<fmt> <ea>,FPn

Syntax: FTWOTOX.X FPm,FPn
FTWOTOX.X FPn

Attributes: Format=(Byte, Word, Long, Single, Double, Extended, Packed)

Description: Converts the source operand to extended precision (if necessary) and calculates two to the power of that number. Stores the result in the destination floating-point data register.

Operation Table:

Destination \ Source	In Range		Zero		Infinity	
	+	-	+	-	+	-
Result	2 ^x		+1.0		+inf	+0.0

NOTE: If the source operand is a NAN, refer to 4.5.4 NANs for more information.

Status Register:

Condition Codes: Affected as described in 4.5.5.1 SETTING FLOATING-POINT CONDITION CODES

Quotient Byte: Not affected

Exception Byte: BSUN Cleared
SNAN Refer to 4.5.4 NANs.
OPERR Cleared
OVFL Refer to 6.1.4 Overflow.
UNFL Refer to 6.1.5 Underflow.
DZ Cleared
INEX2 Refer to 6.1.7 Inexact Result.
INEX1 If <fmt> is Packed, refer to 6.1.8 Inexact Result on Decimal Input; cleared otherwise.

Accrued Exception Byte: Affected as described in 6.1.10 IEEE Exception and Trap Compatibility

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	0	1

Instruction Fields:

Coprocessor ID Field — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Effective Address Field — Determines the addressing mode for external operands.

If R/M=0, this field is unused, and should be all zeros.

If R/M=1, this field is encoded with an M68000 addressing mode as shown:

4

Addressing Mode	Mode	Register
Dn*	000	reg. number:Dn
An	—	—
(An)	010	reg. number:An
(An)+	011	reg. number:An
-(An)	100	reg. number:An
(d16,An)	101	reg. number:An
(dg,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An
([bd,An,Xn],od)	110	reg. number:An
([bd,An],Xn,od)	110	reg. number:An

Addressing Mode	Mode	Register
{xxx}.W	111	000
{xxx}.L	111	001
#<data>	111	100
(d16,PC)	111	010
(dg,PC,Xn)	111	011
(bd,PC,Xn)	111	011
([bd,PC,Xn],od)	111	011
([bd,PC],Xn,od)	111	011

*Only if <fmt> is Byte, Word, Long, or Single.

R/M Field — Specifies the source operand address mode.

0 — The operation is register to register.

1 — The operation is <ea> to register.

Source Specifier Field — Specifies the source register or data format.

If R/M=0, specifies the source floating-point data register, FPN.

If R/M=1, specifies the source data format:

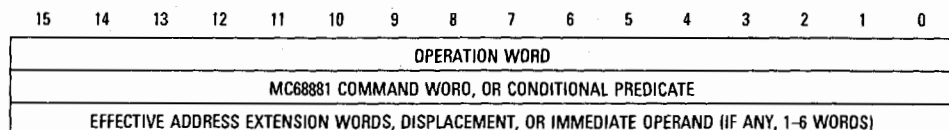
000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Destination Register Field — Specifies the destination floating-point data register, FPN.

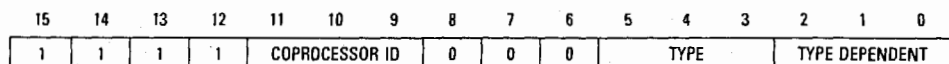
If R/M=0 and the source and destination fields are equal, the input operand is taken from the specified floating-point data register, and the result is written into the same register. If the single register syntax is used, Freescale assemblers set the source and destination fields to the same value.

The following paragraphs provide the details of the object code formats for the general, branch, set on condition, save, and restore type coprocessor instructions.

All FPCP instructions are from two to eight words in length as shown below (the longest case is for an immediate operand of six words — the X or P format).



All FPCP instructions begin with an operation word, formatted as follows:



Coprocessor ID — Specifies which coprocessor in the system is to execute this instruction. Freescale assemblers default to ID=1 for the FPCP.

Type — Specifies the type of coprocessor instruction:

000 — General Instructions (Arithmetics, FMOVE, FMOVEM)

001 — FDBcc, FScC, FTRAPcc

010 — FBcc.W

011 — FBcc.L

100 — FSAVE

101 — FRESTORE

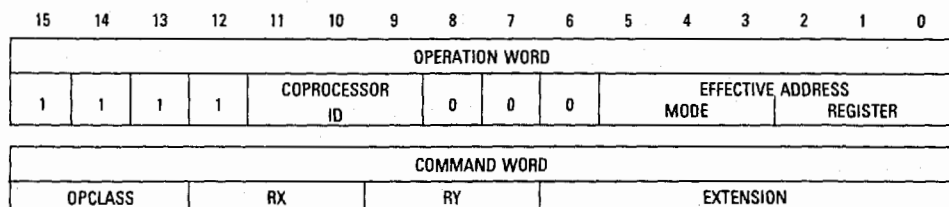
110 — (Undefined, Reserved)

111 — (Undefined, Reserved)

Type Dependent — Normally specifies the effective address or conditional predicate, but usage depends on the type field.

4.7.1 General Type Coprocessor Instruction Format

The general type coprocessor instruction format (shown below) is used for all FPCP arithmetic, move, move multiple, move constant, and transcendental instructions.



The interpretation of the command word fields, OPCLASS, RX, RY, and EXTENSION field varies with the instruction type and is summarized in Table 4-11.

Table 4-11. General Type Instruction Command Word Fields

Opclass	RX	RY	Instruction Class
000	Source, FPM	Destination, FPN	FPM to FPN. The extension field specifies the operation (move, add, etc.)
001	—	—	Undefined, reserved.
010	000-110 Source Data Format	Destination, FPN	Memory to FPN. The extension field specifies the operation (move, add, etc.).
010	111	Destination, FPN	Move constant to FPN. The extension field contains the offset of the ROM constant.
011	Destination Data Format	Source, FPM	Move FPM to an external destination. If the destination format is packed decimal, the extension field specifies the k-factor (#k or Dn); otherwise it should be zero.
100	FPCR Select	000	Move single or multiple to the system control registers. The extension field should be zero.
101	FPCR Select	000	Move single or multiple system control registers to memory. The extension field should be zero.
110	Register list and addressing mode select.	00m	Move multiple to the floating-point data registers. The least significant bit of the RY field and the extension field contains the register list, or the number of the main processor data register that contains the list.
111	Register list and addressing mode select.	00m	Move multiple from the floating-point data registers. The least significant bit of the RY field and the extension field contains the register list, or the number of the main processor data register that contains the list.

The FPCP general type instructions are classified into groups based upon instruction function and argument location (external or internal to the FPCP) as follows:

1. Floating-Point Register to Register
2. External Operand to Floating-Point Data Register
3. Move Constant to Floating-Point Data Register
4. Move Floating-Point Data Register to External Destination
5. Move System Control Register
6. Move Multiple Floating-Point Data Registers

Subdivision of the instruction set on this basis simplifies the specification of the MPU services required by each FPCP instruction. The FPCP requests services from the MPU via the coprocessor interface primitives described in **7.5 INSTRUCTION DIALOGS**.

If the command word indicates that an operand external to the FPCP is to be fetched or stored, the effective address field of the operation word is an MPU effective address descriptor. When the FPCP requests an external data access, the MPU evaluates the source/destination effective address based upon this effective address descriptor and transfers operand(s) to/from the FPCP.

If all operands are contained in FPCP floating-point data registers, the effective address field should be all zeros. If the effective address field is not all zeros, instruction execution proceeds normally; no F-line emulator exception trap is taken. However, to ensure compatibility with future devices, assembler and compiler programmers should fill this field with zeros when it is not used.



REGISTER-TO-REGISTER INSTRUCTIONS. This class of instructions includes g-point data register to floating-point data register moves and the monadic, dyadic arithmetic, and transcendental instructions. For dyadic arithmetic instructions, the destination operand is replaced by the result.

FPm <op> FPn $\rightarrow$ FPn

For monadic arithmetic instructions, the operand is the source FPm and the result is placed into the destination FPn. The source FPm and destination FPn can be the same floating-point data register.

FPm <op> $\rightarrow$ FPn

The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID		0	0	0	0	0	0	0	0	0	0
0	0	0	SOURCE REGISTER		DESTINATION REGISTER		EXTENSION								

Table 4-12 shows the encoding of the source and destination register field.

Table 4-12. Register Field Encoding

000 — FP0	100 — FP4
001 — FP1	101 — FP5
010 — FP2	110 — FP6
011 — FP3	111 — FP7

The extension field indicates the operation to be performed. Table 4-13 lists the extension field encodings and functions. Also shown are the services requested of the MPU by the FPCP.

4.7.1.2 EXTERNAL OPERAND-TO-REGISTER INSTRUCTIONS. This class of instructions includes external operand to floating-point data register move and arithmetic instructions. External operands are located either in memory or an MPU data register (for B, W, L, or S data types). Data format conversion from one of the seven memory data formats to the extended data format is implicit in these instructions. For dyadic arithmetic instructions, the value in FPn is replaced by the result.

External Operand <op> FPn $\rightarrow$ FPn

For monadic arithmetic instructions, the external operand is the source, and the result is placed in the destination FPn.

External Operand <op> $\rightarrow$ FPn

The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	1	0	SOURCE FORMAT			DESTINATION REGISTER			EXTENSION						

Table 4-13. Extension Field Encoding for Arithmetic Operations

Extension Field Services	Instruction Type	MPU
\$00	FMOVE to FpN	Note 1
\$01	FINT	Note 1
\$02	FSINH	Note 1
\$03	FINTRZ	Note 1
\$04	FSQRT	Note 1
\$06	FLOGNP1	Note 1
\$08	FETOXM1	Note 1
\$09	FTANH	Note 1
\$0A	FATAN	Note 1
\$0C	FASIN	Note 1
\$0D	FATANH	Note 1
\$0E	FSIN	Note 1
\$0F	FTAN	Note 1
\$10	FETOX	Note 1
\$11	FTWOTOX	Note 1
\$12	FTENTOX	Note 1
\$14	FLOGN	Note 1
\$15	FLOG10	Note 1
\$16	FLOG2	Note 1

Extension Field Services	Instruction Type	MPU
\$18	FABS	Note 1
\$19	FCOSH	Note 1
\$1A	FNEG	Note 1
\$1C	FACOS	Note 1
\$1D	FCOS	Note 1
\$1E	FGETEXP	Note 1
\$1F	FGETMAN	Note 1
\$20	FDIV	Note 1
\$21	FMOD	Note 1
\$22	FADD	Note 1
\$23	FMUL	Note 1
\$24	FSGLDIV	Note 1
\$25	FREM	Note 1
\$26	FSCALE	Note 1
\$27	FSGLMUL	Note 1
\$28	FSUB	Note 1
\$30-\$37	FSINCOS	Note 1
\$38	FCMP	Note 1
\$3A	FTST	Note 1
\$40-\$7F	(Undefined, Reserved)	Note 2

NOTES:

- Two primitives can be issued for these operations. If the operation is register-to-register, the first primitive issued is null. If any exceptions, other than BSUN, are enabled, PC is set to one to request that the MPU pass the current program counter. If the operation is external operand-to-register, the first primitive is evaluate effective address and transfer data (with CA=1, and PC=1 if any exceptions other than BSUN are enabled). The second primitive is null (CA=0) to terminate the instruction dialog.
- The FPCP issues the take pre-instruction exception primitive with a vector number of 11 to instruct the MPU to take an F-line emulator trap.
- Some extension field encodings are unspecified, are redundant with valid instructions implemented by the FPCP, and do not cause an F-line exception if executed. However, these encodings are reserved for future definition by Motorola, and thus should not be generated by assemblers or compilers. The redundant encodings are: \$05, \$07, \$0B, \$13, \$17, \$1B, \$29-\$2F, \$39, and \$3B-\$3F.

The destination register is encoded as shown in Table 4-12.

The source format field specifies the data format of the external operand. From the external operand are derived the length (in bytes) of the operand and the allowed effective addressing modes. The FPCP decodes the source format field as listed in Table 4-14. The extension field indicates the operation to be performed. Table 4-13 lists the extension field encodings and functions. Also listed are services requested of the MPU by the FPCP.

4.7.1.3 MOVE CONSTANT TO FLOATING-POINT DATA REGISTER INSTRUCTIONS. The FPCP constant ROM contains frequently used constants such as 0.0 and π . These instructions load a correctly rounded constant into a floating-point data register without an external data access.

Table 4-14. Source Format Field Encoding

Source Format Encoding	External Operand Data Format	Length in Bytes	Allowed <ea>
000	Long Word Integer	4	Data
001	Single Precision Real	4	Data
010	Extended Precision Real	12	Memory
011	Packed Decimal Real	12	Memory
100	Word Integer	2	Data
101	Double Precision Real	8	Memory
110	Byte Integer	1	Data

The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	0	0	0	0	0	0
0	1	0	1	1	1	DESTINATION REGISTER		EXTENSION							

The destination register field is encoded as shown in Table 4-12.

The extension field is used as an offset into the FPCP constant ROM. The FMOVECR instruction definition in **4.6 INDIVIDUAL INSTRUCTION DESCRIPTIONS** provides the valid extension field values for the FMOVECR instruction. The only service required by the FPCP from the MPU is the passing of the MPU PC to FPIAR if exceptions (other than BSUN) are enabled. This service is requested with the null (CA = 1, PC = 1) primitive.

4.7.1.4 MOVE TO EXTERNAL DESTINATION INSTRUCTIONS. External destinations are either memory or an MPU data register. Data format conversion from the extended data format to one of the seven memory data formats is implicit for these instructions. The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	1	1	DESTINATION FORMAT			SOURCE REGISTER			EXTENSION						

The source register field is encoded as shown in Table 4-12.

The destination format field indicates the data format of the external destination. The MPU performs all transfers to an external destination at the request of the FPCP. When the FPCP

as a request for a transfer to an external destination, the length (in bytes) of the operand, the allowed effective addressing modes are specified in the primitive.

The FPCP decodes the destination format field to determine the length of the operand to be stored and the allowed effective addressing modes as listed in Table 4-15.

The extension field affects instruction execution only when the destination data format is packed decimal. A destination format encoding of 011 specifies a packed decimal string destination with the formatting parameter, *k*, in the extension field (encoded as a twos-complement value).

Table 4-15. Destination Format Field Encoding

Destination Format Encoding	External Operand Data Format	Length in Bytes	Allowed <ea>
000	Long Word Integer	4	Data Alterable
001	Single Precision Real	4	Data Alterable
010	Extended Precision Real	12	Memory Alterable
011	Packed Decimal Real with Static k-factor	12	Memory Alterable
100	Word Integer	2	Data Alterable
101	Double Precision Real	8	Memory Alterable
110	Byte Integer	1	Data Alterable
111	Packed Decimal Real	12	Memory Alterable with Dynamic k-factor

A destination format encoding of 111 indicates a packed decimal string destination with the formatting parameter, *k*, in an MPU data register. The extension field contains the number of the MPU data register that contains the *k*-factor. The MPU data register number is encoded in bits 6–4 of the extension field; bits 3–0 should be zero. The seven least significant bits of the MPU data register contain a twos-complement *k*-factor. The 25 most significant bits of the MPU data register are ignored. Table 4-16 lists the destination format field encodings, related extension field encodings, instruction operation, and the services requested of the MPU by the FPCP.

4.7.1.5 MOVE SYSTEM CONTROL REGISTER INSTRUCTIONS. This class of instructions includes the move single system control register instruction and the move multiple system control registers instruction. For the move single system control register instruction, external 32-bit operands may be immediate, in memory or an MPU register. For the move multiple system control register instructions, external operands may only be immediate or in memory (immediate addressing is only allowed if *dr*=0). The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
1	0	dr	REGISTER LIST			0	0	0	0	0	0	0	0	0	0

Table 4-16. Extension Field Encoding

Destination Format Encoding	Extension Encoding	External Operand Data Format	MPU Services
000	0000000	Long Word Integer	Notes 1 and 2
001	0000000	Single Precision Real	Notes 1 and 2
010	0000000	Extended Precision Real	Notes 1 and 2
011	kkkkkkk	Packed Decimal Real with a Static k-factor	Note 1
100	0000000	Word Integer	Notes 1 and 2
101	0000000	Double Precision Real	Notes 1 and 2
110	0000000	Byte Integer	Notes 1 and 2
111	rrr0000	Packed Decimal with a Dynamic k-factor	Note 3

NOTES:

- Four service requests can be issued for this instruction type:
 - Null (CA = 1, PC = x) can be first used to request the transfer of the PC to the FPIAR if exceptions are enabled.
 - Null (CA = 1, IA = 1) is used to force the MPU to wait while the conversion takes place.
 - Evaluate effective address and transfer data (CA = 1) is issued to request the transfer of the converted operand.
 - Null (CA = 0) is used to terminate the dialog if no exception occurred. If an exception occurred, the take mid-instruction exception primitive is used to terminate the dialog.
- The extension field should be all zeros, but no F-line emulator trap is taken if it is not. Assemblers and compilers should fill the extension field with zeros to ensure compatibility with future devices.
- Bits 3–0 of the extension field should be zero, but no F-line emulator trap is taken if they are not. Assemblers and compilers should set these bits to zero to ensure compatibility with future devices. Four service requests are issued for this instruction:
 - Transfer single main processor register (CA = 1, PC = x) is first used to request the transfer of the PC to the FPIAR (if exceptions are enabled) and to transfer the MPU data register containing the k-factor.
 - Null (CA = 1, IA = 1) is used to force the MPU to wait while the conversion takes place.
 - Evaluate effective address and transfer data (CA = 1) is issued to request the transfer of the converted operand.
 - Null (CA = 0) is used to terminate the dialog if no exceptions occurred. If an exception occurred, the take mid-instruction exception primitive is used to terminate the dialog.

The dr bit set to one indicates a read of the FPCP; cleared to zero indicates a write to the FPCP. The register select field specifies the system control register or registers to be moved during the operation. Table 4-17 lists the dr and register list field encodings, instruction operation, operand size, allowed effective addressing modes, and services required of the MPU by the FPCP for this instruction type.

Bits 9–0 of the command word should be zero, although no F-line trap is taken if they are not. Assemblers and compilers should set these bits to zeros to ensure compatibility with future devices.

4.7.1.6 MOVE MULTIPLE FLOATING-POINT DATA REGISTERS INSTRUCTIONS. This class of instructions provides move multiple floating-point data register operations analogous to the M68000 move multiple address and data registers instructions. Unlike the integer counterpart, the floating-point register list can be specified either statically in the instruction or dynamically in an MPU data register.

The addressing modes for the move multiple from memory to floating-point data registers instruction are restricted to the control and address register indirect with postincrement effective addressing modes.

The addressing modes for the move multiple from floating-point data registers to memory instruction are restricted to the control alterable and address register indirect with predecrement effective addressing modes.

Table 4-17. Encoding for Move FPCR Operations

Register List	Instruction Operation	Transfer Size (in Bytes)	Allowed <ea>	MC68020/ MC68030 Services
Move Memory to Registers (dr=0)				
000	(Undefined, Reserved)	—	—	Notes 1 and 2
001	Move to FPIAR	4	Any	Note 1
010	Move to FPSR	4	Data	Note 1
011	Move to FPSR and FPIAR	8	Memory	Note 1
100	Move to FPCR	4	Data	Note 1
101	Move to FPCR and FPIAR	8	Memory	Note 1
110	Move to FPCR and FPSR	8	Memory	Note 1
111	Move to FPCR, FPSR, and FPIAR	12	Memory	Note 1
Move Registers to Memory (dr=1)				
000	(Undefined, Reserved)	—	—	Notes 1 and 2
001	Move from FPIAR	4	Alterable	Note 1
010	Move from FPSR	4	Data Alterable 1	
011	Move from FPSR and FPIAR	8	Memory Alterable 1	
100	Move from FPCR	4	Data Alterable 1	
101	Move from FPCR and FPIAR	8	Memory Alterable 1	
110	Move from FPCR and FPSR	8	Memory Alterable 1	
111	Move from FPCR, FPSR, and FPIAR	12	Memory Alterable 1	

NOTES:

1. This operation requires two primitives to be issued to the MPU. The first primitive is evaluate effective address and transfer data (CA=1), indicating the appropriate transfer size and allowed effective addressing mode. The second primitive is null (CA=0) to terminate the instruction dialog.
2. For the current implementation of the FPCR, this encoding is redundant with the 001 encoding of the register select field (i.e., it selects the FPIAR as the only register to be moved); however, this encoding is reserved for future use by Freescale.

NOTE

The effective addressing mode restrictions for this instruction are enforced by the MPU when the transfer multiple coprocessor registers response primitive is received (not by the FPCR when it receives the command word). If the encoding of the effective address field in the operation word is inconsistent with the encoding of the dr and mode fields in the command word, unexpected results occur. In some cases, the instruction is executed, but the order of the register transfer is the reverse of the appropriate order for the addressing mode. However, system integrity is preserved for all cases.

The encoding format for this class of instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODEREGISTER					
1	1	dr	MODE			0	0	0	REGISTER LIST						

bit set to one indicates a read of the FPCP; dr cleared to zero indicates a write to FPCP. The mode field specifies the order of the register transfer and the location of the register list. The definitions of the mode field bits are (bits shown as X may be either zero or one):

- 0X Transfer FP7 through FP0
- 1X Transfer FP0 through FP7
- X0 Register List is Static
- X1 Register List is Dynamic

The order of the register transfer that is selected affects the interpretation of the register list, because the list is always scanned starting with the most significant bit. Thus, for the 0X encoding of the mode field, the most significant bit of the register list corresponds to FP7, and the least significant bit corresponds to FP0. For the 1X encoding, this relationship is reversed.

The type of the register list also affects the interpretation of the register list field. If a static list is selected, then the register list field of the command word contains the register list. If a dynamic register list is selected, then the register list field of the command word contains the number of the MPU data register that contains the register list. The format of the register list field in the command word for the various mode field encodings is shown in the following table. If a bit in the register list is set, then the corresponding register is moved, otherwise the list is scanned for the next bit that is set. For the dynamic register list format, rrr specifies the MPU data register that contains the register list (X means either zero or one). The format of a dynamic register list is the same as the format of the appropriate static list, and it is contained in the least significant eight bits of the MPU data register.

Mode Field
Encoding

Register List Field Format

00	—	FP7	FP6	FP5	FP4	FP3	FP2	FP1	FP0
10	—	FP0	FP1	FP2	FP3	FP4	FP5	FP6	FP7
X1	—	0	r	r	r	0	0	0	0

Table 4-18 lists the dr and mode field encodings, instruction operation, allowed effective addressing modes, and services required of the MPU by the FPCP for this instruction type.

4.7.1.7 UNDEFINED, RESERVED COMMAND WORDS. The command word encoding shown below is undefined and reserved for future Freescale use. All undefined, reserved command word encodings generate an F-line emulator exception.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID				0	0	0	x	x	x	x	x
0	0	1	x	x	x	x	x	x	x	x	x	x	x	x	x

4.7.2 FDBcc, FScC, and FTRAPcc Instruction Formats

For these instruction types, the MPU writes a conditional predicate to the FPCP condition CIR for evaluation. The FPCP determines whether the conditional predicate is true or false

Table 4-18. Encodings for Move Multiple FPN Operations

ode Field	Instruction Operation	Allowed <ea> Modes	MPU Services
Move Memory to Registers (dr=0)			
00	(Invalid Operation)	—	Note 1
01	(Invalid Operation)	—	Note 1
10	Move to Registers, Static Register List	(An)+ or Control	Note 2
11	Move to Registers, Dynamic Register List	(An)+ or Control	Note 3
Move Registers to Memory (dr=1)			
00	Move from Registers, Static Register List	— (An)	Note 2
01	Move from Registers, Dynamic Register List	— (An)	Note 3
10	Move from Registers, Static Register List	Control Alterable	Note 2
11	Move from Registers, Dynamic Register List	Control Alterable	Note 3

NOTES:

- These encodings cause the FPCP to perform an operation that is inconsistent with the M68000 Family move multiple operations. For these cases, the selected registers are transferred in the order that is appropriate for the predecrement addressing mode (i.e., FP7–FP0) using a static or dynamic register list, respectively. However, the MPU does not allow the predecrement addressing mode for a move from memory to multiple coprocessor registers operation. Thus, assemblers and compilers should not generate these encodings, or unexpected results may occur.
- This instruction requires two primitives; the first is the transfer multiple coprocessor registers (CA=1) primitive to request that the MPU evaluate the effective address, read the register select CIR, and transfer the number of registers indicated by the mask (with an operand size of 12 bytes for each register). The second primitive is null (CA=0), which is used to terminate the dialog.
- This instruction requires three primitives; the first is the transfer single main processor register (CA=1) primitive to request the transfer of the MPU data register that contains the dynamic register list. The second is the transfer multiple coprocessor registers (CA=1) primitive to request that the MPU evaluate the effective address, read the register select CIR, and transfer the number of registers indicated by the mask (with an operand size of 12 bytes for each register). The third primitive is null (CA=0) to terminate the dialog.

based on the floating-point condition codes as described in **4.4 CONDITIONAL TEST DEFINITIONS**. The true or false result is returned to the main processor with the null primitive.

These instructions all use the operation word type field encoding and command word format shown below. The instruction specific field of the operation word determines the instruction variation and is defined in Table 4-19 for each instruction type.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	INSTRUCTION SPECIFIC					
0	0	0	0	0	0	0	0	0	0	—	CONDITIONAL PREDICATE				

The conditional predicate field specifies the conditional test to be performed. Table 4-20 lists the conditional predicate encodings and the FPCP responses. For details of the calculation of the response, refer to **4.4 CONDITIONAL TEST DEFINITIONS**. Bits 15–6 of the command word are shown to be filled with zeros; however, no F-line trap is taken if they are not. To ensure compatibility with future devices, assemblers and compilers should fill this field with zeros.

The displacement, extension, or operand words follow immediately after the conditional predicate word. For the FDBcc instruction, the displacement is a 16-bit twos complement integer that indicates the relative distance in bytes from the displacement word (i.e., the

Table 4-19. Encodings for the FDBcc, FScc, and FTRAPcc Instructions

Instruction Specific Field	Instruction Operation	Selected <ea>	MPU Services
000 XXX	FScc <ea>	Dn	Note 1
001 XXX	FDBcc Dn,<label>	—	Note 2
010 XXX	FScc <ea>	(An)	Note 1
011 XXX	FScc <ea>	(An) +	Note 1
100 XXX	FScc <ea>	-(An)	Note 1
101 XXX	FScc <ea>	d ₁₆ (An)	Note 1
110 XXX	FScc <ea>	indexed/indirect	Note 1
111 000	(Undefined, Reserved)	—	Note 3
111 001	(Undefined, Reserved)	—	Note 3
111 010	FTRAPcc.W #<data>	—	Note 4
111 011	FTRAPcc.L #<data>	—	Note 4
111 100	FTRAPcc with No Parameter	—	Note 4
111 101	(Undefined, Reserved)	—	Note 3
111 110	(Undefined, Reserved)	—	Note 3
111 111	(Undefined, Reserved)	—	Note 3

NOTES:

1. The MPU evaluates the <ea> and writes the conditional predicate to the FPCP for evaluation. The null (CA=0) primitive is used to return the true/false evaluation, and the appropriate value is then written to the <ea> by the MPU. The value of XXX specifies the MPU data or address register (Dn or An) used in the <ea> evaluation.
2. The MPU writes the conditional predicate to the FPCP for evaluation. The null (CA=0) primitive is used to return the true/false evaluation. If the condition is true, the MPU proceeds to the next instruction. Otherwise, the counter register Dn.W (specified by the value of XXX) is decremented, and the new value is compared with -1. If it is equal to -1, the MPU proceeds to the next instruction; otherwise, the 16-bit displacement is sign extended and added to the PC.
3. The MPU takes an F-line emulation trap.
4. The MPU writes the conditional predicate to the FPCP for evaluation. The null (CA=0) primitive is used to return the true/false evaluation. If the condition is true, then the cpTRAPcc exception is taken. Otherwise, the MPU proceeds to the next instruction, discarding the optional immediate operand if necessary.

PC value used in the branch destination calculation is the address of the displacement word). For the FScc instruction, the effective address extension words are formatted as required by the main processor. For the FTRAPcc instruction, a one or two word user-defined operand can be included with the instruction. Note that from the perspective of the FPCP, these instructions are identical to the branch type coprocessor instructions. The various operations are handled by the MPU in a manner that is transparent to the FPCP.

4.7.3 Conditional Branch Instruction Format

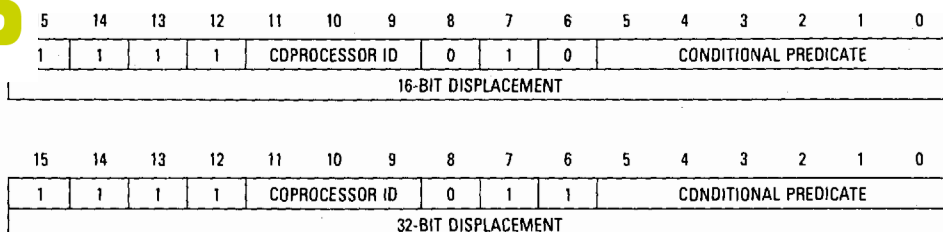
For this instruction type, the MPU writes a conditional predicate to the FPCP condition CIR for evaluation. The FPCP determines whether the conditional predicate is true or false based on the floating-point condition codes as described in 4.4 CONDITIONAL TEST DEFINITIONS. The true or false result is returned to the main processor with the null primitive. The formats for this instruction type are shown below.

Table 4-20. Conditional Predicate Evaluation Responses

Conditional Predicate	Mnemonic	Definition	MC68881/ MC68882 Response
000000	F	False	Note 1
000001	EQ	Equal	Note 1
000010	OGT	Ordered Greater Than	Note 1
000011	OGE	Ordered Greater Than or Equal	Note 1
000100	OLT	Ordered Less Than	Note 1
000101	OLE	Ordered Less Than or Equal	Note 1
000110	OGL	Ordered Greater Than or Less Than	Note 1
000111	OR	Ordered	Note 1
001000	UN	Unordered	Note 1
001001	UEQ	Unordered or Equal	Note 1
001010	UGT	Unordered or Greater Than	Note 1
001011	UGE	Unordered or Greater Than or Equal	Note 1
001100	ULT	Unordered or Less Than	Note 1
001101	ULE	Unordered or Less Than or Equal	Note 1
001110	NE	Not Equal	Note 1
001111	T	True	Note 1
010000	SF	Signaling False	Note 2
010001	SEQ	Signaling Equal	Note 2
010010	GT	Greater Than	Note 2
010011	GE	Greater Than or Equal	Note 2
010100	LT	Less Than	Note 2
010101	LE	Less Than or Equal	Note 2
010110	GL	Greater Than or Less Than	Note 2
010111	GLE	Greater Than or Less Than or Equal	Note 2
011000	NGLE	Not (Greater Than or Less Than or Equal)	Note 2
011001	NGL	Not (Greater Than or Less Than)	Note 2
011010	NLE	Not (Less Than or Equal)	Note 2
011011	NLT	Not (Less Than)	Note 2
011100	NGE	Not (Greater Than or Equal)	Note 2
011101	NGT	Not (Greater Than)	Note 2
011110	SNE	Signaling Not Equal	Note 2
011111	ST	Signaling True	Note 2
1XXXXX	—	(Undefined, Reserved)	Note 3

NOTES:

1. Indicate the condition true or false result by using the null (CA=0) primitive.
2. If the NAN condition code bit is set, then set the BSUN bit in the FPSR. If the BSUN trap is enabled, then return the take pre-instruction exception primitive with the BSUN vector number; otherwise, indicate the condition true/false result by using the null (CA=0) primitive.
3. Not used, redundant encodings with 0XXXXX. No F-line trap is taken if these bit patterns are used. To ensure compatibility with future devices, assemblers and compilers should use the 0XXXXX encodings.



The conditional predicate field specifies the conditional test to be performed. Table 4-20 lists the conditional predicate encodings and the FPCP responses. For details of the response calculation, refer to **4.4 CONDITIONAL TEST DEFINITIONS**.

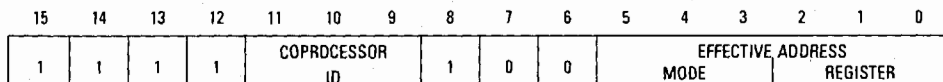
The displacement is a twos-complement integer that indicates the relative distance in bytes from the displacement word(s) (i.e., the PC value used in the branch destination calculation is the address of the displacement word(s)). A 16-bit displacement is sign extended before it is used in the branch destination calculation.

NOTE

From the perspective of the FPCP, the two forms of this instruction are identical. The size of the displacement is determined by the MPU and is transparent to the FPCP. Also, the FNOP instruction syntax that is recognized by Freescale assemblers generates an FBcc.W instruction with cc=F (false) and a displacement value of zero.

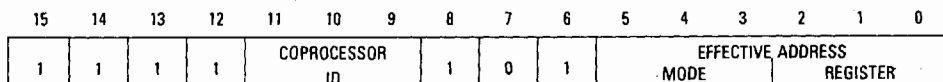
4.7.4 Save Instruction Format

The FSAVE instruction indicates that the FPCP must immediately suspend any current operation and save the internal state in memory. Effective addressing modes are restricted to control alterable and address register indirect with predecrement modes. The encoding format for this instruction is:



4.7.5 Restore Instruction Format

The FPCP restore instruction indicates that regardless of the current state of operation, a new internal state is to be loaded immediately. Effective addressing modes are restricted to control and address register indirect with postincrement modes. The encoding format for this instruction is:



The following paragraphs present a summary of the binary encodings for the FPCP instruction set. The unique encoding for each instruction is shown explicitly, with the encoded fields common to all of the instructions listed in a single table at the beginning of this section.

4

4.8.1 Coprocessor ID Field

This field of each instruction specifies which one of eight (seven, for the MC68030) possible coprocessors in a system is to perform the operation. There are no restrictions placed on the value of the ID field by the main processor in the system; however, certain conventions should be followed. Freescale assemblers default to coprocessor ID = 1 for the FPCP, although directives are available to change this default. Furthermore, due to the hardware implementation of the MC68851 Paged Memory Management Unit, that device must be assigned to coprocessor ID = 0 if used in a system. Thus, the FPCP should not be assigned to coprocessor ID = 0 if it is anticipated that an MC68851 may be used in the system, or in an MC68030 system.

4.8.2 Effective Address Field

This field specifies the M68000 Family addressing mode that is to be used to locate operands external to the FPCP {if required by the instruction). For some operations, restrictions are placed on which of the available addressing modes are allowed. These restrictions are enforced by hardware in the MPU and FPCP, and Freescale assemblers do not generate operation words with disallowed effective addressing mode field encodings. The encodings for this field are shown in Table 4-21.

4.8.3 Register/Memory Field

This field is common to all of the arithmetic instructions and the FMOVE to FPN instruction. A zero in this field indicates that the operation is register-to-register, and a one in this field indicates that the source operand is external to the FPCP.

4.8.4 Source Specifier Field

This field is common to all of the arithmetic instructions and the FMOVE floating-point data register instruction. The definition of this field is affected by the value of the R/M field:

If R/M = 0, it specifies the source floating-point data register, FPM.

If R/M = 1, it specifies the source operand data format:

000	L	Long Word Integer
001	S	Single Precision Real
010	X	Extended Precision Real
011	P	Packed Decimal Real
100	W	Word Integer
101	D	Double Precision Real
110	B	Byte Integer

Table 4-21. Effective Address Field Encoding Summary

Address Modes	Mode	Register	Data	Memory	Control	Alterable	Assembler Syntax
Data Register Direct	000	reg. no.	X	—	—	X	Dn
Address Register Direct	001	reg. no.	—	—	—	X	An
Address Register Indirect	010	reg. no.	X	X	X	X	(An)
Address Register Indirect with Postincrement	011	reg. no.	X	X	—	X	(An) +
Address Register Indirect with Predecrement	100	reg. no.	X	X	—	X	— (An)
Address Register Indirect with Displacement	101	reg. no.	X	X	X	X	(d ₁₆ ,An)
Address Register Indirect with Index (8-Bit Displacement)	110	reg. no.	X	X	X	X	(dg,An,Xn)
Address Register Indirect with Index (Base Displacement)	110	reg. no.	X	X	X	X	(bd,An,Xn)
Memory Indirect Postindexed	110	reg. no.	X	X	X	X	{(bd,An),Xn,od}
Memory Indirect Preindexed	110	reg. no.	X	X	X	X	{(bd,An,Xn),od}
Absolute Short	111	000	X	X	X	X	(xxx).W
Absolute Long	111	001	X	X	X	X	(xxx).L
Program Counter Indirect with Displacement	111	010	X	X	X	—	(d ₁₆ ,PC)
Program Counter Indirect with Index (8-Bit Displacement)	111	011	X	X	X	—	(dg,PC,Xn)
Program Counter Indirect with Index (Base Displacement)	111	011	X	X	X	—	(bd,PC,Xn)
PC Memory Indirect Postindexed	111	011	X	X	X	—	{(bd,PC),Xn,od}
PC Memory Indirect Preindexed	111	011	X	X	X	—	{(bd,PC,Xn),od}
Immediate	111	100	X	X	—	—	#<data>

4.8.5 Destination Register Field

This field is common to all of the arithmetic instructions and the FMOVE to FPN instruction. This field specifies the floating-point data register that is to be used as the destination. The result of an operation is always stored in this register, and one of the source operands is fetched from this register for dyadic instructions.

4.8.6 Conditional Predicate Field

This field is common to all of the conditional instructions and specifies the FPCP conditional test that is to be evaluated for the main processor. Table 4-22 shows the conditional predicate binary encodings for the 32 conditional tests supported by the FPCP.

Table 4-22. Conditional Predicate Field Encoding Summary

Conditional Predicate	Mnemonic	Definition
000000	F	False
000001	EQ	Equal
000010	OGT	Ordered Greater Than
000011	OGE	Ordered Greater Than or Equal
000100	OLT	Ordered Less Than
000101	OLE	Ordered Less Than or Equal
000110	OGL	Ordered Greater Than or Less Than
000111	OR	Ordered
001000	UN	Unordered
001001	UEQ	Unordered or Equal
001010	UGT	Unordered or Greater Than
001011	UGE	Unordered or Greater Than or Equal
001100	ULT	Unordered or Less Than
001101	ULE	Unordered or Less Than or Equal
001110	NE	Not Equal
001111	T	True
010000	SF	Signaling False
010001	SEQ	Signaling Equal
010010	GT	Greater Than
010011	GE	Greater Than or Equal
010100	LT	Less Than
010101	LE	Less Than or Equal
010110	GL	Greater Than or Less Than
010111	GLE	Greater Than or Less Than or Equal
011000	NGLE	Not (Greater Than or Less Than or Equal)
011001	NGL	Not (Greater Than or Less Than)
011010	NLE	Not (Less Than or Equal)
011011	NLT	Not (Less Than)
011100	NGE	Not (Greater Than or Equal)
011101	NGT	Not (Greater Than)
011110	SNE	Signaling Not Equal
011111	ST	Signaling True

The instruction formats are summarized in this section.

FMOVE to FPn

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	0	0

4

FINT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	0	1

FSINH

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	1	0

FINTRZ

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	0	1	1

FSQRT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	1	0	0

FLOGNP1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	0	1	1	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	0	0	0

FTANH

4

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	0	0	1

FATAN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	0	1	0

FASIN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	0	0

FATANH

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	0	1

FSIN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	1	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	0	1	1	1	1

FETOX

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	0	0

FTWOTOX

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	0	1

FTENTOX

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	0	1	0

FLOGN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	CDPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	0	0

FLOG10

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	0	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	0	1	1	0

FABS

4

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	0	0

FCOSH

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	C
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	0	1

FNEG

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	C
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	0	1	C

FACOS

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	0	0

FCOS

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	0	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	1	0

FGETMAN

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	0	1	1	1	1	1

4

FDIV

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	0	0

FMOD

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	0	1

FADD

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	1	0

FMUL

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	0	1	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	0	0

FREM

4

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	0	1

FSCALE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	1	0

FSGLMUL

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	0	1	1	1

FSUB

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	0	1	0	0	0

FSINCOS

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE			ADDRESS REGISTER		
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER, FPs			0	1	1	0	DESTINATION REGISTER, FPs		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	1	1	0	0	0

FTST

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	R/M	0	SOURCE SPECIFIER			DESTINATION REGISTER			0	1	1	1	0	1	0

4

FMOVECR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	0	0	0	0	0	0
0	1	0	1	1	1	DESTINATION REGISTER			ROM OFFSET						

ROM Offset Field — Specifies the offset in the FPCP Constant ROM where the desired constant is located.

FMOVE from FPn

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
0	1	1	DESTINATION FORMAT			SOURCE REGISTER			K-FACTOR (IF REQUIRED)						

Destination Format Field — Specifies the data format of the destination operand as follows:

- 000 — Long Word Integer
- 001 — Single Precision Real
- 010 — Extended Precision Real
- 011 — Packed Decimal Real, static k-factor
- 100 — Word Integer
- 101 — Double Precision Real
- 110 — Byte Integer
- 111 — Packed Decimal Real, dynamic k-factor

k-factor Field — Specifies the format of the packed decimal string to be generated (if the destination format field indicates packed decimal), or the number of the main processor data register that contains the format specification. The interpretation of the k-factor is:

- 64 to 0 — Number of significant digits to the right of the decimal point.
- +1 to +17 — Number of significant digits in the mantissa.
- +18 to +63 — Sets the OPERR bit, treated as +17.

The format of this field for a dynamic k-factor is:

rrr0000

Where rrr is the number of the main processor data register that contains the k-factor.



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
1	0	dr	REGISTER SELECT			0	0	0	0	0	0	0	0	0	0

dr Field — Specifies the direction of the transfer:

0 — Move memory to system control register

1 — Move system control register to memory

Register Select Field — Specifies the system control register to be moved:

001 — FPIAR

010 — FPSR

100 — FPCR

4

FMOVEM FPcr

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE		REGISTER			
1	0	dr	REGISTER LIST			0	0	0	0	0	0	0	0	0	0

dr Field — Specifies the direction of the transfer:

0 — Move memory to system control registers

1 — Move system control registers to memory

Register List Field — Specifies the system control registers to be moved:

000 — (Undefined, reserved)

100 — FPCR

001 — FPIAR

101 — FPCR, then FPIAR

010 — FPSR

110 — FPCR, then FPSR

011 — FPSR, then FPIAR

111 — FPCR, then FPSR, then FPIAR

FMOVEM FPn

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	0	EFFECTIVE ADDRESS MODE REGISTER					
1	1	dr	MODE		0	0	0	REGISTER LIST							

dr Field — Specifies the direction of the transfer

0 — Move the listed registers from memory to the FPCP

1 — Move the listed registers from the to memory

Mode Field — Specifies the type of the register list and addressing mode

00 — Static register list, predecrement addressing mode

01 — Dynamic register list, predecrement addressing mode

10 — Static register list, postincrement or control addressing mode

11 — Dynamic register list, postincrement or control addressing mode

Register List Field:

Static list — contains the select mask; if a register is to be moved, the corresponding bit in the list is set, otherwise it is clear.



amic list — contains the main processor data register number, rrr, as shown below:

Register List Format

Static, $-(An)$	—	FP7	FP6	FP5	FP4	FP3	FP2	FP1	FP0
Static, $(An)+$ or Control	—	FP0	FP1	FP2	FP3	FP4	FP5	FP6	FP7
Dynamic	—	0	r	r	r	0	0	0	0

The format of the dynamic list mask is the same as for the static list and is contained in the least significant eight bits of the specified MPU data register.

FScc

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	EFFECTIVE ADDRESS MODE REGISTER					
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					

FDBcc

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	0	0	1	COUNT REGISTER		
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					
16-BIT DISPLACEMENT															

Count Register Field — Specifies the main processor data register to be decremented

FTRAPcc

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	0	1	1	1	1	MODE		
0	0	0	0	0	0	0	0	0	0	CONDITIONAL PREDICATE					
16-BIT OPERAND OR MOST SIGNIFICANT WORD OF 32-BIT OPERAND (IF NEEDED)															
LEAST SIGNIFICANT WORD OF 32-BIT OPERAND (IF NEEDED)															

Mode Field — Specifies the form of the instruction:

010 — The instruction is followed by a 16-bit operand.

011 — The instruction is followed by a 32-bit operand.

100 — The instruction has no operand following it.

FNOP

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			0	1	SIZE	CONDITIONAL PREDICATE					
16-BIT DISPLACEMENT, OR MOST SIGNIFICANT WORD OF 32-BIT DISPLACEMENT															
LEAST SIGNIFICANT WORD OF 32-BIT DISPLACEMENT (IF NEEDED)															

Size Field — Specifies the size of the twos-complement displacement:

Size=0 — Displacement is 16-bits (and is sign extended before it is used).

Size=1 — Displacement is 32-bits.

4

FSAVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	0	EFFECTIVE ADDRESS MODE REGISTER					

FRESTORE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	COPROCESSOR ID			1	0	1	EFFECTIVE ADDRESS MODE REGISTER					